EpaDash Backend Technical Requirements and Core System Architecture

Document ID	EPADASH-BE-ARCH-001
Version	0.2 — Revised after technical review
Status	Submitted for approval (revision 2)
Prepared	8 November 2026
Scope	Original Assignment Task 5

STATUS: SUBMITTED FOR APPROVAL — REVISION 2. This specification responds to the technical review feedback. Company standards, providers, hosting and final business rules remain subject to technical review; the baseline decisions in Section 2.3 are recorded so that approval can be given on a concrete foundation.

> **Revision note:** Version 0.1 was returned with the feedback that database relationships, API capabilities, payment processing and delivery management were described only conceptually or deferred to later work. This revision specifies each of them in implementable detail: complete entity and relationship definitions with cardinalities and delete rules, an endpoint catalogue with role access and request and response structures, the payment processing design and the delivery management design. Open provider and policy choices have been replaced with explicit baseline decisions (Section 2.3) so that approval can be given on a concrete foundation. Section 1.2 maps each review point to the sections that address it.

Contents

1 Executive summary **4**
- 1.1 Revision history . . . 4
- 1.2 Review feedback resolution . . . 4

2 Assignment scope, assumptions and decisions **5**
- 2.1 In-scope backend domains . . . 5
- 2.2 Confirmed requirements from the assignment . . . 5
- 2.3 Design decisions and assumptions . . . 6

3 Technology stack and database strategy **7**
- 3.1 Technology stack . . . 7
- 3.2 Database strategy . . . 8

4 Architecture style and deployment shape **8**
- 4.1 Recommended style: modular monolith . . . 9
- 4.2 Logical request flow . . . 9
- 4.3 Layer responsibilities . . . 9
- 4.4 Runtime components . . . 11
- 4.5 Cross-cutting design . . . 11

5 Logical module boundaries **12**

6 Database design and relationships **13**
- 6.1 Modelling conventions . . . 13
- 6.2 Entity catalogue . . . 13
- 6.3 Relationships and delete behaviour . . . 19
- 6.4 Entity relationship diagrams . . . 20
- 6.5 Integrity and concurrency rules . . . 20
- 6.6 Indexing strategy . . . 22

6.7 Migrations, seed data and data lifecycle . . . 23
7 Roles and access control 24
7.1 Roles . . . 24
7.2 Permission matrix . . . 24
7.3 Authorisation rules . . . 26
8 Authentication and account security 26
8.1 Authentication model . . . 26
8.2 Required flows . . . 27
8.3 Password and credential handling . . . 27
8.4 Account and approval statuses . . . 29
9 API requirements and conventions 30
9.1 General conventions . . . 30
9.2 Common response shape . . . 30
9.3 Error codes . . . 31
9.4 Rate limits . . . 32
9.5 Endpoint catalogue . . . 32
9.6 Core request and response structures . . . 37
10 Core business workflows 40
10.1 Merchant and product discovery . . . 40
10.2 Order creation . . . 41
10.3 Order status lifecycle . . . 42
10.4 Payment processing . . . 44
10.5 Delivery management . . . 48
10.6 Notifications . . . 52
10.7 Administration and audit . . . 53
11 Third-party integration requirements 54
11.1 Integration summary . . . 54
11.2 Adapter contracts . . . 55
11.3 Payment gateway requirements . . . 55
11.4 Mapping requirements . . . 56
11.5 Notification requirements . . . 56
11.6 File and media storage . . . 56
11.7 Configuration and secrets . . . 57
12 Security, privacy and operational requirements 57
12.1 Security and privacy . . . 57
12.2 Non-functional targets . . . 58
12.3 Environments and delivery process . . . 58
13 Testing and quality strategy 59
14 Proposed repository structure 60
15 Implementation roadmap 61
16 Delivery and approval checklist — Task 5 61

List of Figures

1 System architecture context: request path, domain modules, provider adapters and external services. 10
2 Logical request flow through the backend layers. . . . 10
3 Entity relationships: identity, merchants and catalogue. Link tables, option groups and staff records are described in the entity catalogue. . . . 21
4 Entity relationships: orders and order items. . . . 21
5 Entity relationships: payments, ledger and delivery. The ledger also references orders, merchants and riders, and webhook events are matched to payments by provider reference. . . . 22
6 Login flow. . . . 28
7 Refresh-token rotation and reuse detection. . . . 28
8 Password recovery and reset sequence. . . . 29
9 Order status lifecycle. . . . 43

10 Online payment initiation and confirmation sequence. 45
11 Payment webhook verification and idempotent processing. 46
12 Rider dispatch and assignment sequence. 50
13 Delivery status lifecycle. 52
14 End-to-end workflow from order creation to delivery completion. 54

List of Tables

1 Revision history 4
2 Resolution of review feedback 4
3 Baseline design decisions 6
4 Backend technology stack 7
5 Runtime components 11
6 Logical module boundaries 12
7 Entities: identity and access 14
8 Entities: role profiles 14
9 Entities: merchants, catalogue and inventory 15
10 Entities: orders 16
11 Entities: payments and money 17
12 Entities: delivery 17
13 Entities: notifications, messaging and audit 18
14 Relationship matrix 19
15 Principal indexes 23
16 User roles and intended access 24
17 Permission matrix (own = only resources belonging to the principal) 25
18 Authentication parameters 26
19 Account status transitions 29
20 API conventions 30
21 Error codes 31
22 Default rate limits 32
23 Authentication endpoints 32
24 User and profile endpoints 33
25 Merchant discovery and catalogue endpoints (public) 33
26 Merchant management endpoints 34
27 Order and payment endpoints (customer) 35
28 Rider and delivery endpoints 35
29 Notification endpoints 36
30 Administration and system endpoints 36
31 Order status transitions 42
32 Payment status transitions 44
33 Ledger entries 47
34 Payment failure handling 48
35 Delivery status transitions 48
36 Delivery configuration defaults (platform settings) 51
37 Delivery exceptions 52
38 Notification events 52
39 Third-party integrations 54
40 Principal configuration variables 57
41 Initial non-functional targets 58
42 Implementation phases 61
43 Task 5 acceptance criteria and evidence 61
44 Approval record 62

1 Executive summary

EpaDash is a delivery platform on which customers order from merchants, merchants prepare orders, riders deliver them and administrators oversee the platform. The backend must therefore support customer, merchant, rider and administrator management; merchant and product discovery; order processing; payments; delivery management; notifications; and administration.

The recommended design is a **modular monolith** backed by PostgreSQL and exposed through a versioned REST API. The implementation stack is Node.js with TypeScript, Express.js, PostgreSQL, Prisma ORM, Zod validation, OpenAPI 3.1 documentation and Vitest/Supertest testing. A separate worker process, sharing the same code base, runs background jobs for notifications, payment reconciliation, order timeouts and rider dispatch retries.

This document defines the technology stack, system architecture, core modules, the complete data model with relationships, the role and permission model, authentication and authorisation, the API surface, the order, payment and delivery workflows, and the payment, mapping and notification integrations. It is intended to be sufficient for the detailed design and implementation tasks that follow, without those tasks having to take further architectural decisions.

1.1 Revision history

Table 1. Revision history

Version	Date	Description
0.1	7 November 2026	Initial draft: stack, architecture style, module list, conceptual entity outline. Returned after review.
0.2	8 November 2026	Revised after review: full data model with relationships, complete API catalogue, payment processing design, delivery management design, concrete integration decisions, roadmap and traceability.

1.2 Review feedback resolution

Table 2. Resolution of review feedback

Review concern	Resolution in this version	Sections
Database relationships remain conceptual	Entity catalogue of 36 tables with attributes, keys and constraints; relationship matrix with cardinality, foreign key and delete behaviour; three entity relationship diagrams; integrity, concurrency, indexing and data-lifecycle rules	Section 6
API capabilities deferred to a later task	API conventions, response and error model, rate limits, a complete endpoint catalogue per role and domain, and request and response structures for the core endpoints	Section 9

Review concern	Resolution in this version	Sections
Payment processing not specified	Payment model and state machine, initiation and webhook sequences, reconciliation, refunds, ledger and settlement, cash on delivery, failure handling and security controls	Section 10.4, Section 11
Delivery management not specified	Delivery state machine, dispatch algorithm with concurrency control, tracking, proof of delivery, fees and earnings, failure and reassignment rules	Section 10.5
Document not comprehensive enough as a foundation	Decision register replacing open items, permission matrix, implementation roadmap and acceptance-criteria traceability	Section 2.3, Section 7, Section 15, Section 16

2 Assignment scope, assumptions and decisions

2.1 In-scope backend domains

1. **Customer management:** customer accounts, profiles, addresses, account status and customer-facing access.
2. **Merchant management:** merchant accounts and public merchant profiles, categories, operating hours, operating status and delivery availability.
3. **Rider management:** rider accounts, verification and status information, availability and delivery participation.
4. **Product management:** product catalogue, categories, options, publication state, price, availability and inventory.
5. **Order processing:** order creation, order items, pricing snapshots and the order status lifecycle.
6. **Delivery management:** delivery assignment, rider association, delivery status, tracking events and proof of delivery.
7. **Payments:** payment initiation and reference, provider verification, transaction status, refunds, ledger and reconciliation records.
8. **Notifications:** notification events and delivery through configured channels.
9. **Administration:** privileged account and platform operations, and audit records.
10. **Cross-cutting capabilities:** authentication, authorisation, validation, consistent errors, logging, security, API documentation, tests and database migrations.

2.2 Confirmed requirements from the assignment

- Merchant listing, search, category filtering, profile retrieval, operating status and delivery availability.
- Product catalogue retrieval, product search, category filtering, details, pricing and availability.
- Public discovery must not expose unpublished products or merchant listings that are not active and intended for public display.

- Registration, login, logout, password recovery and reset, session and token management, and role-based access.
- Customer, merchant, rider and administrator roles, profile management, account status, verification requirements and permission enforcement.
- Relational database entities, relationships, constraints, indexes, migrations and seed scripts.
- Order, payment, rider assignment and delivery workflows.
- API contracts for authentication, customer, merchant, product, order, payment, rider, delivery, notification and administration domains.
- Payment, mapping and notification integration requirements.

2.3 Design decisions and assumptions

The assignment does not name a payment gateway, mapping provider, messaging provider, hosting platform, expected traffic or jurisdiction-specific policies. Rather than leave these open, the decisions in Table 3 are adopted as the baseline. Each decision is isolated behind an interface, a configuration value or a single module, so replacing it later does not change the architecture, the data model or the workflows in this document. Where the organisation mandates a different standard, that standard replaces the corresponding decision.

Table 3. Baseline design decisions

ID	Topic	Decision
D1	Technology stack	Node.js LTS, TypeScript (strict), Express.js, PostgreSQL, Prisma ORM, Zod, OpenAPI 3.1, Vitest and Supertest (Section 3).
D2	Architecture	Modular monolith: one API process and one worker process built from the same code base. No microservices initially.
D3	Payment gateway	Paystack hosted checkout for cards and mobile money, behind a `PaymentProvider` adapter. Cash on delivery is supported as an internal payment method.
D4	Mapping provider	Google Maps Platform (Geocoding and Routes/Distance Matrix) behind a `MappingProvider` adapter, with a straight-line (Haversine) fallback.
D5	Notifications	In-app notifications plus push (Firebase Cloud Messaging), SMS and email through `NotificationProvider` adapters; delivery through a queue with retry.
D6	Currency and money	One currency per deployment (ISO 4217 code in configuration); amounts stored as `NUMERIC(12,2)` in major units and converted to minor units only when calling the gateway.
D7	Background jobs	PostgreSQL-backed job queue (pg-boss) run by the worker process; no additional infrastructure. Redis may be added later for caching and distributed rate limiting.
D8	Tokens and sessions	Short-lived JWT access token and rotating opaque refresh token stored hashed server-side (Section 8.1).
D9	Hosting	Containerised deployment with a managed PostgreSQL service and S3-compatible object storage; platform choice is not mandated by the architecture.

ID	Topic	Decision
D10	Real-time behaviour	Version 1 uses REST polling for order and delivery tracking together with push notifications. WebSocket or server-sent events are a defined extension point.
D11	Stock rule	Stock is reserved when an order is created, committed when the order is placed and released on cancellation, rejection or payment timeout (Section 10.2).
D12	Dispatch	Automatic nearest-available rider dispatch with administrator manual override (Section 10.5).
D13	Roles	One role per user account: CUSTOMER, MERCHANT, RIDER, ADMIN or SUPER_ADMIN.

3 Technology stack and database strategy

3.1 Technology stack

Table 4. Backend technology stack

Layer	Choice	Reason / notes
Runtime	Node.js LTS	Mature server-side JavaScript runtime
Language	TypeScript, strict mode	Compile-time type checking and maintainability
HTTP framework	Express.js	Small, flexible REST framework; layers are kept explicit
Database	PostgreSQL 15 or later	Relational integrity, transactions, row locking, partial indexes and JSONB
ORM / migrations	Prisma ORM and Prisma Migrate	Typed database access; migrations plus reviewed raw SQL for features Prisma does not model
Validation	Zod	Runtime validation of request payloads, responses and environment configuration
API contract	OpenAPI 3.1	Generated from the Zod schemas so the contract cannot drift from the code
API explorer	Swagger UI	Human-readable interactive API reference
Background jobs	pg-boss	Queue stored in PostgreSQL; retries, scheduling and locking without new infrastructure
HTTP hardening	helmet, cors, express-rate-limit	Secure headers, origin allow-list and rate limiting
Tests	Vitest + Supertest	Unit and HTTP integration testing against a real PostgreSQL test database
Logging	Pino	Structured logs with redaction of secrets and sensitive fields

Layer	Choice	Reason / notes
Password hashing	Argon2id	Password-specific one-way hashing
Object storage	S3-compatible storage	Product and merchant images, rider documents
Source control	Git + GitHub	Branching, review and traceability
Containers	Docker and Docker Compose	Repeatable local PostgreSQL environment and deployable images

The final decision on the stack is recorded in an Architecture Decision Record (ADR-0001). If a company-mandated stack exists, that standard overrides this proposal.

3.2 Database strategy

- **Engine and layout:** one PostgreSQL primary database shared by all modules, in a single schema. Each table is owned by exactly one module (Table 6).
- **Schema management:** Prisma Migrate with forward-only, reviewed migrations. Constraints that Prisma cannot express (partial unique indexes, CHECK constraints, trigram indexes, immutability triggers) are added as hand-written SQL inside the same migrations.
- **Identifiers and time:** UUID primary keys for business entities and `BIGINT` identity keys for high-volume log tables. All timestamps are `TIMESTAMPTZ` stored in UTC; local time is a presentation concern.
- **Money:** `NUMERIC(12,2)` in major units with an explicit currency code; never floating point.
- **Deletion:** soft delete (`deleted_at`) for users, merchants and products so that order, payment and audit history is never orphaned. Financial and audit tables are append-only.
- **Transactions:** every multi-row business operation (order creation, stock reservation, payment confirmation, delivery assignment) runs in one interactive Prisma transaction. Contended rows are locked with `SELECT ... FOR UPDATE` (or `SKIP LOCKED` for queues).
- **Search and geography:** trigram (`pg_trgm`) indexes for name search; latitude and longitude columns with bounding-box filtering and Haversine distance. PostGIS is the documented upgrade path if radius or polygon queries outgrow this.
- **Environments:** local development and automated tests use Docker Compose PostgreSQL; staging and production use a managed instance with automated backups (Section 12).
- **Seed data:** reference data (roles, categories, configuration defaults) is seeded in every environment; sample data exists only in development and test.

4 Architecture style and deployment shape

4.1 Recommended style: modular monolith

One backend application is deployed as a single service. Internally it is divided into domain modules with clear interfaces. Modules share one PostgreSQL database but must not reach into another module's internal implementation or tables; cross-module interaction occurs through exported service interfaces or domain events.

Why this is the recommended starting point

- The platform is a connected system with transactional relationships across users, merchants, products, orders, payments and deliveries.
- One service simplifies local setup, deployment, debugging and end-to-end testing.
- Database transactions can be used directly for operations that must be atomic.
- Module boundaries preserve maintainability and leave room for later extraction if measurable needs arise.

Microservices are not recommended as the initial default because there is no traffic, scaling, independent release or organisational requirement that justifies distributed deployment complexity.

Figure 1 shows the architecture context, including the domain modules described in Section 5.

4.2 Logical request flow

Client application → API router → authentication middleware → request validation → controller → application/service layer → repository/data access → PostgreSQL. External provider interactions are performed by dedicated adapters. Responses use a common success and error envelope (Section 9.2). Figure 2 illustrates the flow.

4.3 Layer responsibilities

- **Routes:** bind HTTP method and path to middleware and controller; no business rules.
- **Middleware:** authentication, permission checks, request IDs, rate limiting and common request concerns.
- **Controllers:** translate HTTP requests into application commands and map results to HTTP responses.
- **Services / use cases:** enforce business rules and coordinate transactions.
- **Repositories:** encapsulate database reads and writes; always scoped by the authenticated principal where ownership applies.
- **Provider adapters:** wrap payment, mapping and messaging provider APIs behind the interfaces in Section 11.2.
- **Schemas:** validate request, response and environment configuration.
- **Tests:** verify domain rules, API behaviour, security boundaries and integration contracts.

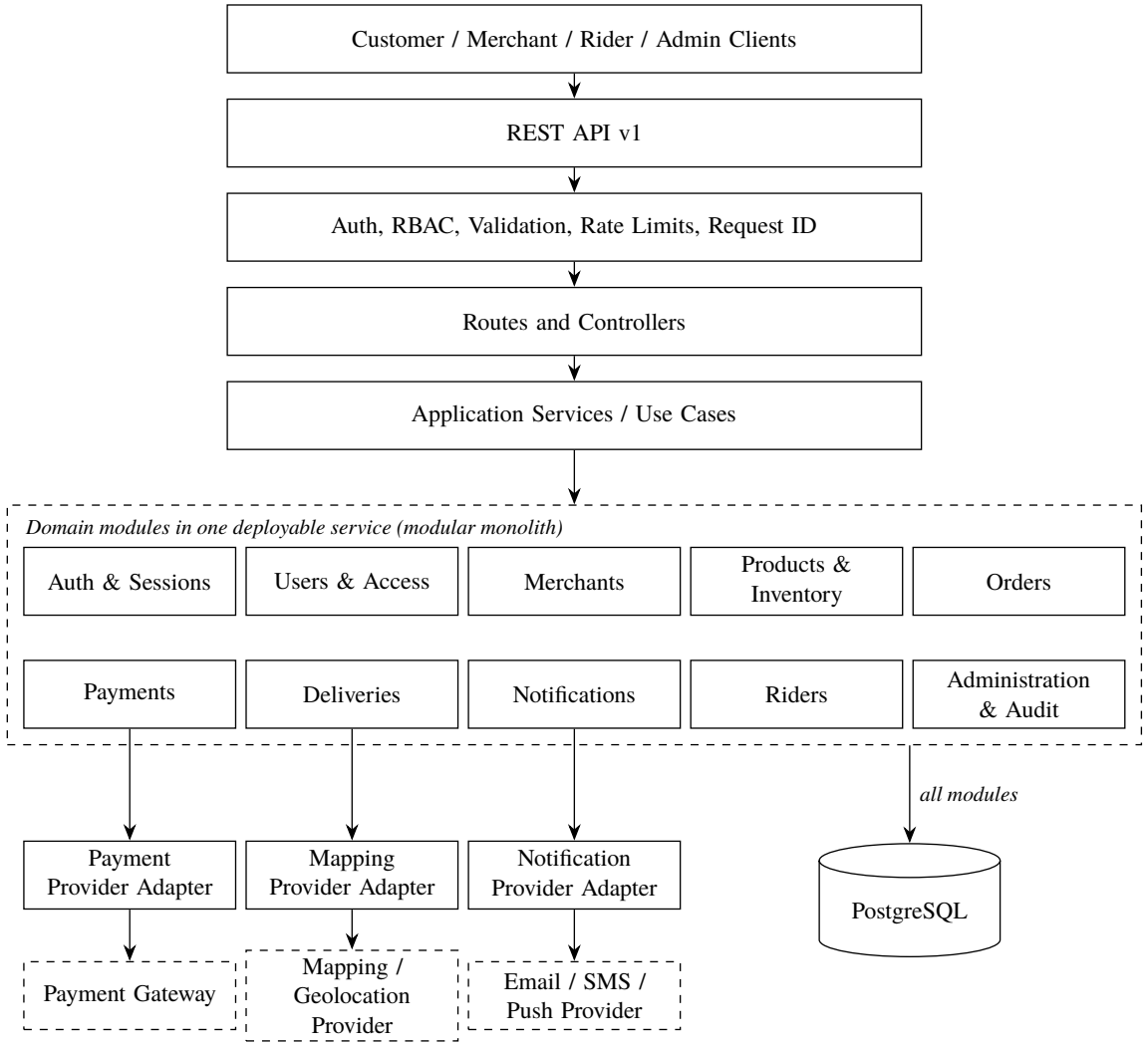


Figure 1. System architecture context: request path, domain modules, provider adapters and external services.

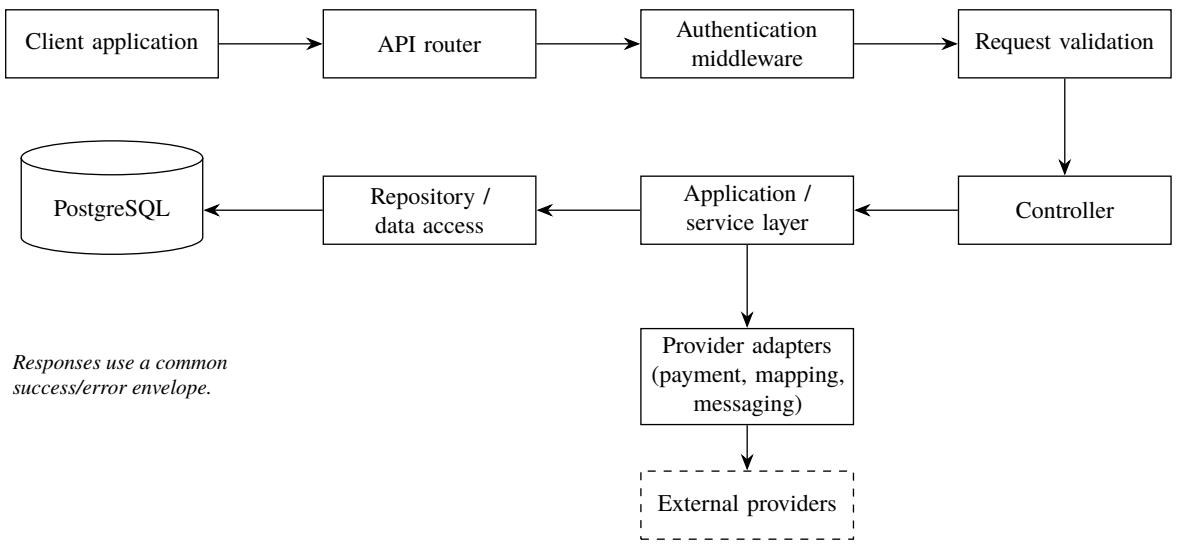


Figure 2. Logical request flow through the backend layers.

4.4 Runtime components

Table 5. Runtime components

Component	Responsibility	Scaling
API process	Serves the REST API; stateless; verifies access tokens and enforces authorisation	Horizontal, behind a load balancer
Worker process	Consumes pg-boss queues: notification delivery, outbox dispatch, payment reconciliation, order and payment timeouts, dispatch retries, data-retention jobs	One or more instances; jobs are locked so each runs once
PostgreSQL	System of record, transactional state and job queue	Vertical first; read replicas later if required
Object storage	Images and rider documents, accessed through presigned URLs	Provider managed
External providers	Payment gateway, mapping, push, SMS and email	Provider managed

4.5 Cross-cutting design

- **Error handling:** services throw typed application errors that a single error middleware maps to the standard error envelope; unexpected errors return a generic `INTERNAL_ERROR` and are logged with the request ID.
- **Request tracing:** every request receives a request ID, returned in the response and attached to every log line.
- **Idempotency:** order creation and payment initiation require an `Idempotency-Key` header, enforced by unique constraints; payment webhooks are deduplicated by provider event ID.
- **Domain events and outbox:** state changes write an event row to `outbox_events` in the same transaction; the worker turns events into notifications and audit entries, so a notification failure can never corrupt order or payment state.
- **Audit:** privileged and sensitive actions write immutable `audit_logs` rows.
- **Configuration:** all configuration is read once at start-up, validated with Zod, and the process refuses to start on invalid configuration.

5 Logical module boundaries

Table 6. Logical module boundaries

Module	Responsibilities	Owned tables	Primary dependencies
Auth	Registration, login, logout, refresh, password reset, verification, sessions	sessions, otp_codes, password_reset_tokens	Users, notifications
Users & Access	Accounts, roles, profiles, addresses, account status, permissions	roles, users, customers, admins, addresses	Auth, audit
Merchants	Merchant profile and approval, categories, hours, delivery zones, staff, public visibility	categories, merchants, merchant_staff, merchant_ categories, merchant_hours, delivery_zones	Users, products
Products	Products, options, publication state, pricing	products, product_option_ groups, product_options	Merchants, inventory
Inventory	Stock levels, reservations, adjustments, movement history	inventory_items, inventory_ movements	Products, orders
Orders	Quote and order validation, order items, pricing snapshot, lifecycle	orders, order_items, order_ status_history	Users, merchants, products, inventory, payments, deliveries
Payments	Payment initiation, verification, webhooks, reconciliation, refunds, ledger	payments, payment_ webhook_events, refunds, transactions	Orders, payment adapter, audit
Riders	Rider profile, documents, verification, availability and location	riders, rider_documents	Users
Deliveries	Delivery creation, dispatch, assignment, status, tracking, proof of delivery	deliveries, delivery_ assignments, delivery_ tracking	Orders, riders, mapping adapter
Notifications	Notification creation, preferences, device tokens, channel dispatch, outbox processing	notifications, notification_ preferences, device_tokens, outbox_events	Users, messaging adapters
Administration & Audit	Privileged operations, approvals, reporting, audit trail	audit_logs	All modules through approved interfaces

Dependency rules: a module may call another module only through its exported service interface; it never writes to a table it does not own. The module list describes the target platform scope; modules may be delivered in stages (Section 15) without changing these boundaries.

6 Database design and relationships

This section defines the complete relational model: every entity, its principal attributes, its keys and constraints, and every relationship with its cardinality and delete behaviour. Enumerated values are shown in upper case and are implemented as PostgreSQL enum types (Prisma enums).

6.1 Modelling conventions

- Primary keys are named `id` (UUID), except for one-to-one profile tables whose primary key is the owning `user_id` or `product_id`, and for log tables that use a `BIGINT` identity.
- Foreign keys are named `<entity>_id` and are always declared; every foreign key column is indexed.
- Tables and columns use `snake_case` in the database and are mapped to `camelCase` in the Prisma client.
- Every mutable table has `created_at` and `updated_at`; `updated_at` is maintained by Prisma or a trigger.
- Order items preserve the product name, unit price and selected options at purchase time, and orders preserve a snapshot of the delivery address, so later edits never change history.
- Statuses are enumerations with explicit transition rules enforced in the service layer (Section 8.4, Section 10.3, Section 10.4, Section 10.5).

6.2 Entity catalogue

Identity and access

Table	Key attributes	Keys and constraints

Table 7. Entities: identity and access

Table	Key attributes	Keys and constraints
`roles`	id (smallint), name (CUSTOMER, MERCHANT, RIDER, ADMIN, SUPER_ADMIN), description	PK id; name unique
`users`	id, role_id, email (case-insensitive), phone (E.164), password_hash, status (PENDING_ VERIFICATION, ACTIVE, SUSPENDED, DISABLED), email_verified_at, phone_verified_ at, failed_login_count, locked_until, last_login_at, status_reason, deleted_at	FK role_id; email unique; phone unique; CHECK email or phone present; index (role_id, status)
`sessions`	id, user_id, family_id, refresh_token_hash, user_ agent, ip_address, expires_at, revoked_at	FK user_id; refresh_token_hash unique; index family_id; index user_id where not revoked
`otp_codes`	id, user_id, channel (EMAIL, SMS), purpose (VERIFY_EMAIL, VERIFY_PHONE), code_ hash, attempts, expires_at, consumed_at	FK user_id; index (user_id, purpose, created_at)
`password_reset_ tokens`	id, user_id, token_hash, expires_at, used_at	FK user_id; token_hash unique
`addresses`	id, user_id, label, recipient_name, phone, line1, line2, city, region, country_code, latitude, longitude, delivery_notes, is_default	FK user_id; one default per user (partial unique index); latitude and longitude range checks

Role profiles

Table 8. Entities: role profiles

Table	Key attributes	Keys and constraints
`customers`	user_id, first_name, last_name, avatar_url	PK and FK user_id
`admins`	user_id, first_name, last_name	PK and FK user_id
`riders`	user_id, first_name, last_name, vehicle_type (BICYCLE, MOTORBIKE, CAR), vehicle_plate, national_id_encrypted, verification_status (PENDING, APPROVED, REJECTED), verified_ by, verified_at, availability (OFFLINE, AVAILABLE, BUSY), current_latitude, current_ longitude, last_location_at	PK and FK user_id; partial index (availability, last_location_at) for approved riders

Table	Key attributes	Keys and constraints
rider_documents	id, rider_id, doc_type (NATIONAL_ID, DRIVERS_LICENSE, VEHICLE_ REGISTRATION, SELFIE), file_key (private storage), status, reviewed_by, reviewed_at	FK rider_id; FK reviewed_by

Merchants, catalogue and inventory

Table 9. Entities: merchants, catalogue and inventory

Table	Key attributes	Keys and constraints
categories	id, parent_id, type (MERCHANT, PRODUCT), name, slug, image_url, sort_order, is_active	self FK parent_id; unique (type, slug)
merchants	id, owner_user_id, business_name, slug, description, logo_url, cover_image_url, phone, email, status (PENDING_APPROVAL, ACTIVE, SUSPENDED, CLOSED), is_open, accepts_ delivery, accepts_pickup, min_order_amount, avg_prep_minutes, address fields, latitude, longitude, timezone (IANA name), commission_ rate, rating_avg, rating_count, approved_by, approved_at, deleted_at	FK owner_user_id; slug unique; partial index for public listing; trigram index on business_name; index (latitude, longitude)
merchant_staff	merchant_id, user_id	composite PK; FKs to merchants and users
merchant_ categories	merchant_id, category_id	composite PK; resolves the many-to-many link
merchant_hours	id, merchant_id, day_of_week (0 to 6, 0 = Sunday), opens_at, closes_at	unique (merchant_id, day_of_week, opens_at); a closing time earlier than the opening time means the period runs past midnight
delivery_zones	id, merchant_id, name, max_distance_km, delivery_fee, est_minutes, is_active	FK merchant_id; CHECK max_ distance_km > 0
products	id, merchant_id, category_id, name, slug, description, image_url, sku, price, compare_at_ price, currency, status (DRAFT, PUBLISHED, ARCHIVED), is_available, track_inventory, sort_ order, deleted_at	FKs merchant_id, category_id; unique (merchant_id, slug); CHECK price >= 0; partial index for public listing; trigram index on name
product_option_ groups	id, product_id, name, min_select, max_select, sort_order	FK product_id; CHECK max_select >= min_select
product_options	id, group_id, name, price_delta, is_available, sort_order	FK group_id

Table	Key attributes	Keys and constraints
inventory_items	product_id, quantity_on_hand, quantity_reserved, low_stock_threshold	PK and FK product_id; CHECK on_hand >= 0, reserved >= 0 and reserved <= on_hand
inventory_movements	id (bigint), product_id, change_qty, reason (RESTOCK, SALE, RESERVATION, RELEASE, ADJUSTMENT, RETURN, WASTE), order_id, created_by, note	FKs product_id, order_id; append-only; index (product_id, created_at)

Orders

Table 10. Entities: orders

Table	Key attributes	Keys and constraints
orders	id, order_number, customer_id, merchant_id, status (PENDING_PAYMENT, PLACED, ACCEPTED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, COMPLETED, CANCELLED, REJECTED), fulfilment (DELIVERY, PICKUP), payment_method (CARD, MOBILE_MONEY, BANK_TRANSFER, CASH_ON_DELIVERY), payment_status, delivery_address_id, delivery_address (JSONB snapshot), delivery latitude and longitude, currency, subtotal, delivery_fee, service_fee, discount_total, tax_total, total, customer_note, cancellation_reason, cancelled_by, idempotency_key, expires_at, placed_at, accepted_at, ready_at, completed_at, cancelled_at	FKs customer_id, merchant_id, delivery_address_id; order_number unique; unique (customer_id, idempotency_key); CHECK delivery orders carry an address; indexes (customer_id, created_at), (merchant_id, status, created_at), (status, created_at)
order_items	id, order_id, product_id, product_name, unit_price, quantity, selected_options (JSONB), line_total, note	FKs order_id, product_id; CHECK quantity > 0
order_status_history	id (bigint), order_id, from_status, to_status, changed_by, note, created_at	FK order_id; append-only; index (order_id, created_at)

Payments and money

Table 11. Entities: payments and money

Table	Key attributes	Keys and constraints
`payments`	id, order_id, provider, provider_reference, method, amount, currency, status (PENDING, SUCCESS, FAILED, CANCELLED, REFUNDED, PARTIALLY_REFUNDED), idempotency_key, gateway_response (sanitised JSONB), failure_reason, paid_at	FK order_id; provider_reference unique; idempotency_key unique; partial unique index allowing one SUCCESS payment per order
`payment_webhook_events`	id (bigint), provider, event_id, event_type, payload (JSONB), processed_at	unique (provider, event_id) for idempotent processing
`refunds`	id, payment_id, amount, reason, status (PENDING, SUCCESS, FAILED), provider_reference, requested_by	FK payment_id; CHECK amount > 0
`transactions`	id, reference, type (CHARGE, REFUND, PLATFORM_FEE, MERCHANT_PAYOUT, RIDER_PAYOUT, COD_COLLECTION), direction (CREDIT, DEBIT), amount, currency, status, order_id, payment_id, refund_id, merchant_id, rider_id	reference unique; immutable ledger (UPDATE and DELETE blocked by trigger)

Delivery

Table 12. Entities: delivery

Table	Key attributes	Keys and constraints
`deliveries`	id, order_id, rider_id, status (UNASSIGNED, ASSIGNED, AT_PICKUP, PICKED_UP, DELIVERED, FAILED, CANCELLED), pickup and drop-off coordinates, distance_km, est_minutes, delivery_fee, rider_earning, confirmation_code_hash, confirmation_attempts, proof_photo_url, failure_reason, assigned_at, picked_up_at, delivered_at	order_id unique; FK rider_id (null until assigned); CHECK a rider is present once assigned; indexes (rider_id, status) and unassigned by age
`delivery_assignments`	id, delivery_id, rider_id, status (OFFERED, ACCEPTED, REJECTED, EXPIRED, CANCELLED), offered_at, expires_at, responded_at, reason	FKs delivery_id, rider_id; partial unique index allowing one OFFERED row per delivery
`delivery_tracking`	id (bigint), delivery_id, rider_id, latitude, longitude, heading, speed_kmh, accuracy_m, recorded_at	FK delivery_id; index (delivery_id, recorded_at descending)

Notifications, messaging and audit

Table 13. Entities: notifications, messaging and audit

Table	Key attributes	Keys and constraints
`notifications`	id, user_id, channel (IN_APP, PUSH, SMS, EMAIL), type, title, body, data (JSONB deep-link payload), status (PENDING, SENT, FAILED), attempts, sent_at, read_at, error	FK user_id; index (user_id, created_at); partial index on unread in-app items
`notification_preferences`	user_id, channel, category (ORDERS, PROMOTIONS, ACCOUNT), enabled	composite PK (user_id, channel, category)
`device_tokens`	id, user_id, token, platform (ANDROID, IOS, WEB), last_used_at	token unique; FK user_id
`audit_logs`	id (bigint), actor_user_id, actor_role, action, entity_type, entity_id, old_values, new_values, ip_address, user_agent, created_at	FK actor_user_id; append-only; indexes (entity_type, entity_id), (actor_user_id, created_at)
`outbox_events`	id (bigint), event_type, aggregate_type, aggregate_id, payload (JSONB), status (PENDING, PROCESSED, FAILED), attempts, created_at, processed_at	partial index on PENDING rows for the worker

Relationship (parent to child)	Cardinality	Foreign key	On delete

6.3 Relationships and delete behaviour

Table 14. Relationship matrix

Relationship (parent to child)	Cardinality	Foreign key	On delete
roles to users	1 : N	users.role_id	RESTRICT
users to sessions, otp_codes, password_reset_tokens	1 : N	user_id	CASCADE
users to addresses	1 : N	addresses.user_id	CASCADE
users to customers, riders, admins	1 : 1	user_id (also PK)	CASCADE
users to merchants (owner)	1 : N	merchants.owner_user_id	RESTRICT
merchants to users (staff)	N : M	merchant_staff (merchant_id, user_id)	CASCADE
merchants to categories	N : M	merchant_categories (merchant_id, category_id)	CASCADE
categories to categories (parent)	1 : N	categories.parent_id	SET NULL
merchants to merchant_hours, delivery_zones	1 : N	merchant_id	CASCADE
merchants to products	1 : N	products.merchant_id	CASCADE (soft delete in practice)
categories to products	1 : N	products.category_id	SET NULL
products to product_option_groups	1 : N	product_id	CASCADE
product_option_groups to product_options	1 : N	group_id	CASCADE
products to inventory_items	1 : 0..1	inventory_items.product_id	CASCADE
products to inventory_movements	1 : N	product_id	CASCADE
customers to orders	1 : N	orders.customer_id	RESTRICT
merchants to orders	1 : N	orders.merchant_id	RESTRICT
addresses to orders	1 : N	orders.delivery_address_id	SET NULL (snapshot retained)
orders to order_items	1 : N (at least 1)	order_items.order_id	CASCADE

Relationship (parent to child)	Cardinality	Foreign key	On delete
products to order_items	1 : N	order_items.product_id	SET NULL (snapshot retained)
orders to order_status_history	1 : N	order_id	CASCADE
orders to payments	1 : N	payments.order_id	RESTRICT
payments to refunds	1 : N	refunds.payment_id	RESTRICT
orders, payments, refunds, merchants, riders to transactions	1 : N each	optional foreign keys on transactions	RESTRICT
orders to deliveries	1 : 0..1	deliveries.order_id	CASCADE
riders to deliveries	1 : N	deliveries.rider_id (null until assigned)	RESTRICT
deliveries to delivery_ assignments	1 : N	delivery_id	CASCADE
riders to delivery_assignments	1 : N	rider_id	RESTRICT
deliveries to delivery_tracking	1 : N	delivery_id	CASCADE
riders to rider_documents	1 : N	rider_id	CASCADE
users to notifications, notification_preferences, device_tokens	1 : N	user_id	CASCADE
users to audit_logs (actor)	1 : N	actor_user_id	RESTRICT

Financial, order and audit records use RESTRICT or SET NULL so they can never be removed by deleting a parent; users, merchants and products are soft-deleted.

6.4 Entity relationship diagrams

Figure 3, Figure 4 and Figure 5 show the entities and the cardinalities of the relationship matrix. Dashed boxes are entities shown for context. Link tables, lookup details and log tables are described in Section 6.2.

6.5 Integrity and concurrency rules

1. An account must have an email address or a phone number, and both are unique when present.
2. Prices, fees, totals and quantities cannot be negative; an order item quantity must be at least 1.
3. Stock on hand and reserved stock cannot become negative and reserved stock cannot exceed stock on hand. Reservations use a conditional update (`UPDATE ... SET quantity_reserved = quantity_reserved + n WHERE quantity_on_hand - quantity_reserved >= n`) and fail when no row is updated, so two customers cannot buy the last item.

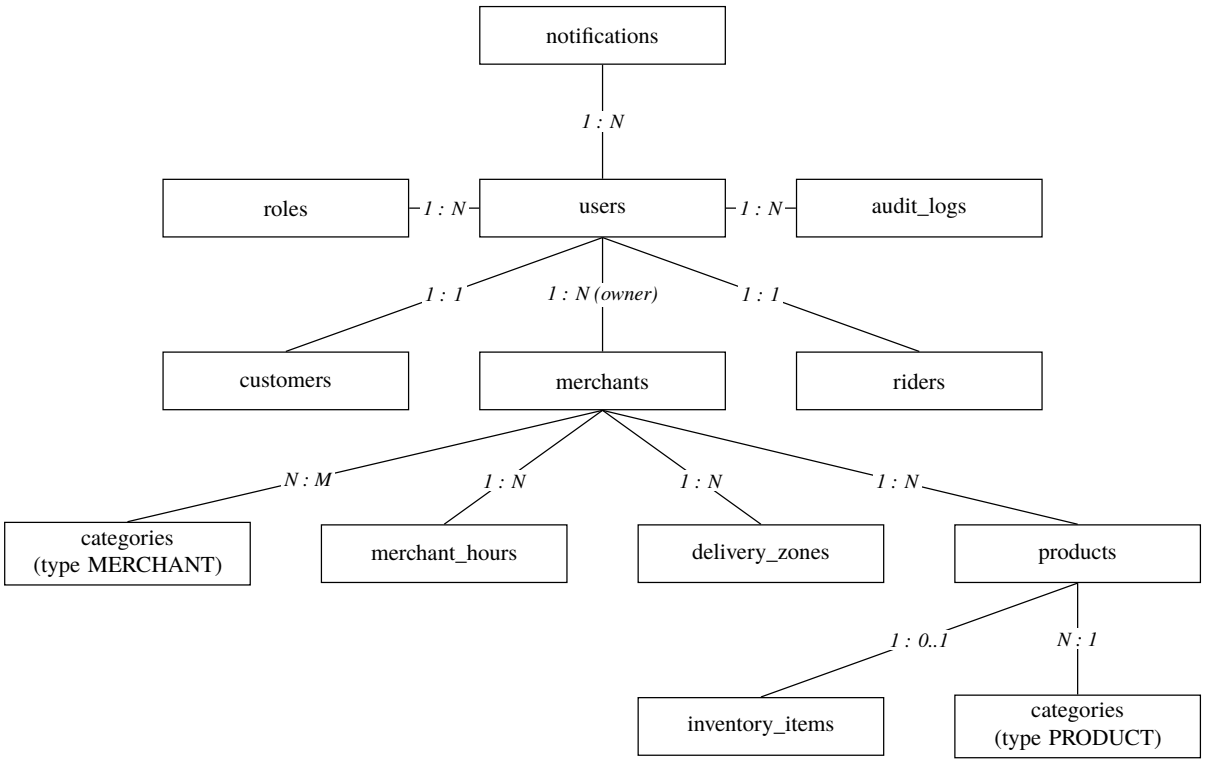


Figure 3. Entity relationships: identity, merchants and catalogue. Link tables, option groups and staff records are described in the entity catalogue.

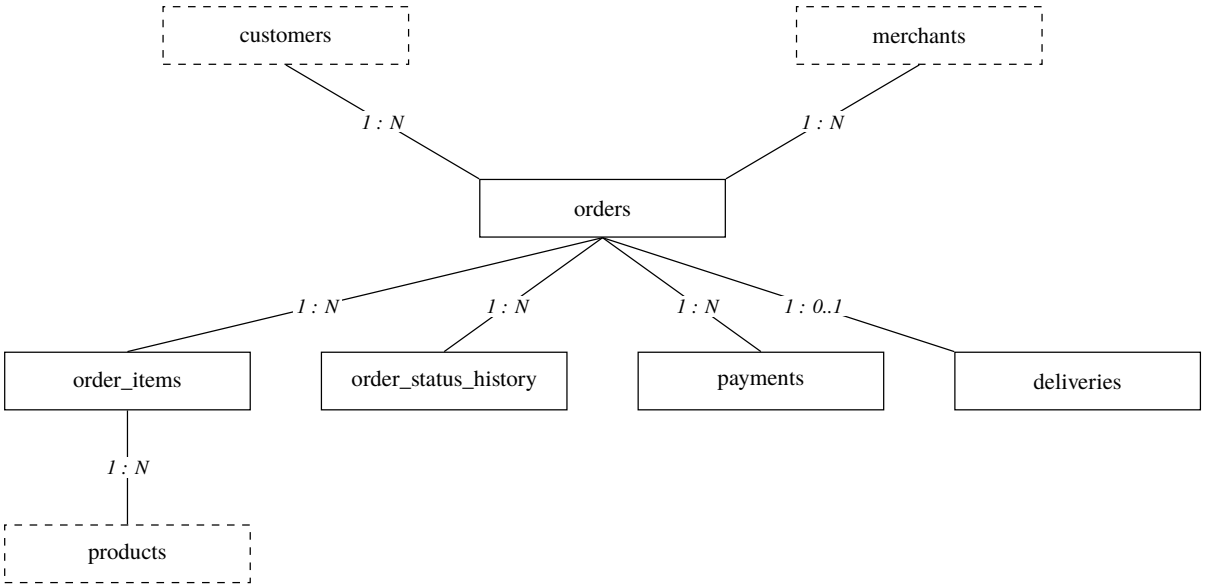


Figure 4. Entity relationships: orders and order items.

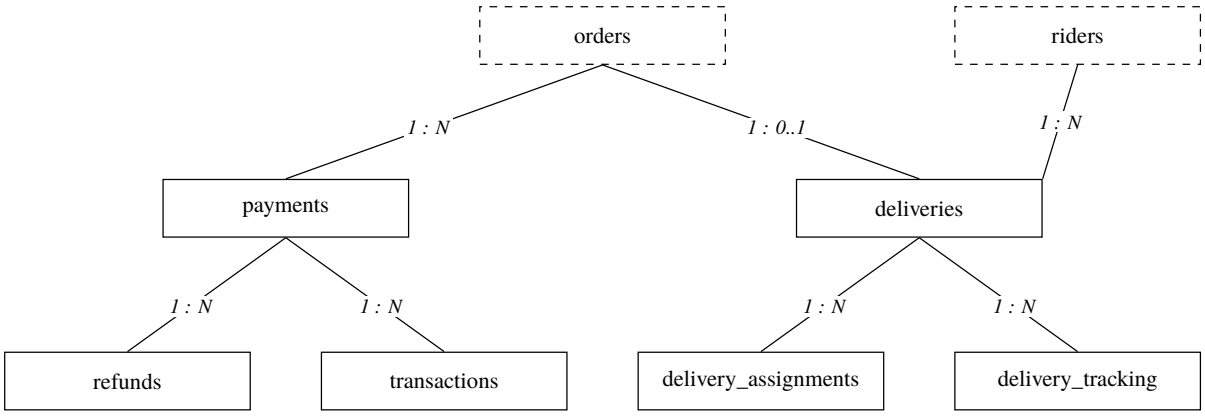


Figure 5. Entity relationships: payments, ledger and delivery. The ledger also references orders, merchants and riders, and webhook events are matched to payments by provider reference.

4. An order can have only one successful payment (partial unique index), only one delivery (unique `order_id`) and a delivery can have only one open rider offer (partial unique index).
5. Duplicate checkouts, payment initiations and webhooks are rejected or replayed through unique idempotency keys and event identifiers.
6. Delivery assignment locks the delivery row (`SELECT ... FOR UPDATE`) so that exactly one rider can win an offer, and queue consumers claim work with `FOR UPDATE SKIP LOCKED`.
7. Order, payment and delivery status changes are made only through service methods that check the current status inside the transaction (compare-and-set), write a history row and write an outbox event.
8. The `transactions` ledger and `audit_logs` are append-only; triggers reject UPDATE and DELETE.
9. Order creation, payment confirmation and delivery completion are each a single database transaction; partial results are never committed.
10. Public queries always apply the visibility predicate of Section 10.1; no public query reads the base tables without it.

6.6 Indexing strategy

Table	Index	Purpose

Table 15. Principal indexes

Table	Index	Purpose
merchants	partial (city, is_open, rating_avg) where ACTIVE and not deleted	Public merchant listing and sorting
merchants	GIN trigram on business_name; (latitude, longitude)	Name search; bounding-box distance filter
products	partial (merchant_id, category_id, sort_ order) where PUBLISHED, available and not deleted	Public catalogue by merchant and category
products	GIN trigram on name	Product search
orders	(customer_id, created_at desc); (merchant_id, status, created_at desc); (status, created_at)	Customer history, merchant queue, administrator dashboards and timeout jobs
order_items	order_id; product_id	Order detail; product sales queries
payments	order_id; (status, created_at); unique provider_reference	Payment lookup; reconciliation of pending payments
transactions	order_id; (merchant_id, created_at); (rider_ id, created_at)	Settlement and earnings reports
deliveries	(rider_id, status); partial created_at where UNASSIGNED	Rider workload; dispatch retry queue
delivery_ assignments	partial rider_id where OFFERED; (delivery_ id, offered_at desc)	Open offers per rider; offer history
delivery_tracking	(delivery_id, recorded_at desc)	Latest location and trail
riders	partial (availability, last_location_at) where APPROVED	Dispatch candidate search
sessions	(user_id) where not revoked; family_id; expires_at	Session list, reuse detection, expiry clean-up
notifications	(user_id, created_at desc); partial pending; partial unread in-app	Inbox, worker queue, unread badge
audit_logs	(entity_type, entity_id, created_at); (actor_ user_id, created_at)	Audit search

6.7 Migrations, seed data and data lifecycle

- **Migrations:** one Prisma migration per change, reviewed in pull requests, applied automatically in CI and by a controlled release step in staging and production. Hand-written SQL for partial indexes,

CHECK constraints, trigram indexes and immutability triggers lives in the same migration.

- **Reference seed:** roles, merchant and product categories and configuration defaults; idempotent and run in every environment.
- **Development seed:** sample users for every role, merchants (including a pending merchant), products (including a draft and a sold-out product) and stock levels, so discovery visibility rules can be tested. Never run in production.
- **Retention defaults (configurable):** expired sessions, OTP codes and reset tokens purged daily after seven days; rider location points older than 90 days deleted or aggregated; processed webhook events and outbox rows archived after 12 months; orders, payments, ledger and audit records retained according to the organisation's financial and legal policy.

7 Roles and access control

7.1 Roles

Table 16. User roles and intended access

Role	Intended access
Customer	Manage own profile and addresses; browse public active merchants and published products; create, pay for, view and cancel (while permitted) own orders; track own deliveries; manage own notification settings
Merchant	Manage own merchant profile, hours, delivery zones and open status; manage own catalogue and inventory; view and process orders belonging to that merchant; view own settlement information. Staff accounts act within one merchant only
Rider	Manage own profile, documents and availability; send location updates; view and respond to own delivery offers; update milestones on assigned deliveries; view own earnings
Administrator	Approve, suspend and manage users, merchants and riders; view all orders, payments and deliveries; reassign deliveries; process refunds; manage categories and platform configuration; view reports and audit logs. SUPER_ADMIN additionally creates administrators and changes platform configuration

7.2 Permission matrix

Resource and action	Customer	Merchant	Rider	Admin

Table 17. Permission matrix (own = only resources belonging to the principal)

Resource and action	Customer	Merchant	Rider	Admin
Browse active merchants and published products	Yes	Yes	Yes	Yes
View and edit own profile and password	Own	Own	Own	Own
Manage saved addresses	Own	No	No	No
Create and manage merchant profile, hours, zones	No	Own	No	Any
Approve, suspend or close a merchant	No	No	No	Yes
Create, edit, publish and archive products	No	Own	No	Any
Adjust inventory	No	Own	No	Any
Create an order and request a quote	Yes	No	No	No
View orders	Own	Own merchant	Assigned only	Any
Cancel an order before it is accepted	Own	No	No	Any
Accept, reject, prepare, mark ready	No	Own merchant	No	Any
Initiate payment	Own order	No	No	No
View payment status	Own	Own merchant (summary)	No	Any
Issue a refund	No	No	No	Yes
Set availability and send location	No	No	Own	No
Accept or reject a delivery offer	No	No	Own	No
Update delivery milestones	No	No	Assigned only	Any
Assign or reassign a rider	No	No	No	Yes
Approve or reject a rider	No	No	No	Yes
View own notifications and preferences	Own	Own	Own	Own
Suspend or reactivate a user	No	No	No	Yes
Create administrators; change platform settings	No	No	No	SUPER_ ADMIN
View reports and audit logs	No	Own merchant (reports)	Own (earnings)	Yes

7.3 Authorisation rules

- Deny by default; explicitly grant only the permissions required.
- Authentication proves identity; authorisation decides whether the identity may perform the requested action.
- Enforce permissions on the server for every protected endpoint through a middleware chain: authenticate, check account status, check role, then enforce resource ownership.
- Enforce resource ownership and tenant boundaries in addition to role checks. A merchant must not access another merchant's private resources; a customer must not access another customer's order; a rider must not update another rider's assignment. Ownership is enforced by repositories that scope every query by the authenticated principal (for example `merchant_id IN (merchants of this user)`), so an unauthorised identifier returns not found.
- Never trust a role, user ID, merchant ID or ownership claim supplied only in a request body; derive identity from the authenticated principal and verify resource relationships.
- Administrative privileges are narrowly defined, and every administrative change records the actor, the reason and the time in the audit log.

8 Authentication and account security

8.1 Authentication model

The system uses short-lived signed access tokens and rotating refresh tokens. For browser clients the tokens are delivered in `Secure`, `HttpOnly`, `SameSite` cookies with CSRF protection on state-changing requests; native and mobile clients send the access token in the `Authorization` header and keep the refresh token in the platform's secure storage.

Table 18. Authentication parameters

Parameter	Specification
Access token	Signed JWT (HS256, secret from the secret store), lifetime 15 minutes; claims `sub` (user ID), `role`, `sid` (session ID) and `jti`
Refresh token	Opaque 256-bit random value, lifetime 30 days, rotated on every use; only its SHA-256 hash is stored in `sessions`
Reuse detection	Presenting a refresh token that was already rotated revokes the whole token family (`family_id`) and ends the session
Session limit	At most 10 active sessions per user; the oldest is revoked when exceeded
Password hashing	Argon2id with memory cost of at least 19 MiB, time cost 2 and parallelism 1, tuned to the deployment hardware
Password policy	Minimum 8 and maximum 128 characters; check against a list of common and breached passwords; no composition rules

Parameter	Specification
Login protection	5 consecutive failures lock the account for 15 minutes (`locked_until`); additional IP and identifier rate limits (Table 22)
Verification code	6 digits, valid for 10 minutes, at most 5 attempts, 60 second resend interval; stored hashed
Password reset token	256-bit random value, valid for 30 minutes, single use, stored hashed
Transport	HTTPS only outside local development; HSTS enabled

8.2 Required flows

1. **Registration:** validate fields, normalise login identifiers, hash the password with Argon2id, create the account in `PENDING_VERIFICATION` status and send a verification code. Customers become `ACTIVE` after verification; merchant and rider accounts additionally require administrator approval (Section 8.4).
2. **Login:** verify credentials, check account status, issue an access token and a refresh token with a new session record, and record the security-relevant event (Figure 6).
3. **Logout:** revoke the refresh token and session; the access token's `jti` is rejected until it expires.
4. **Refresh:** rotate the refresh token, detect reuse and revoke the compromised token family and session (Figure 7).
5. **Password recovery:** return a non-enumerating response; send a single-use, expiring reset token through the configured channel (Figure 8).
6. **Password reset:** validate the token, enforce the password policy, update the password hash and revoke all existing sessions and refresh tokens.
7. **Account activation and suspension:** enforce status checks at authentication and on protected operations; record the actor, reason and time of administrative changes.

8.3 Password and credential handling

- Store only password hashes, never plaintext or reversible passwords.
- Use Argon2id with parameters reviewed against deployment resources and current security guidance.
- Store secrets in environment or secret management, never in source control.
- Do not log passwords, access tokens, refresh tokens, reset tokens, payment secrets or full sensitive payloads.
- Apply rate limits and abuse controls to login, registration, password recovery and verification endpoints.
- Use TLS for all non-local traffic.
- Return the same response and similar timing for existing and non-existing accounts on login failure and password recovery.

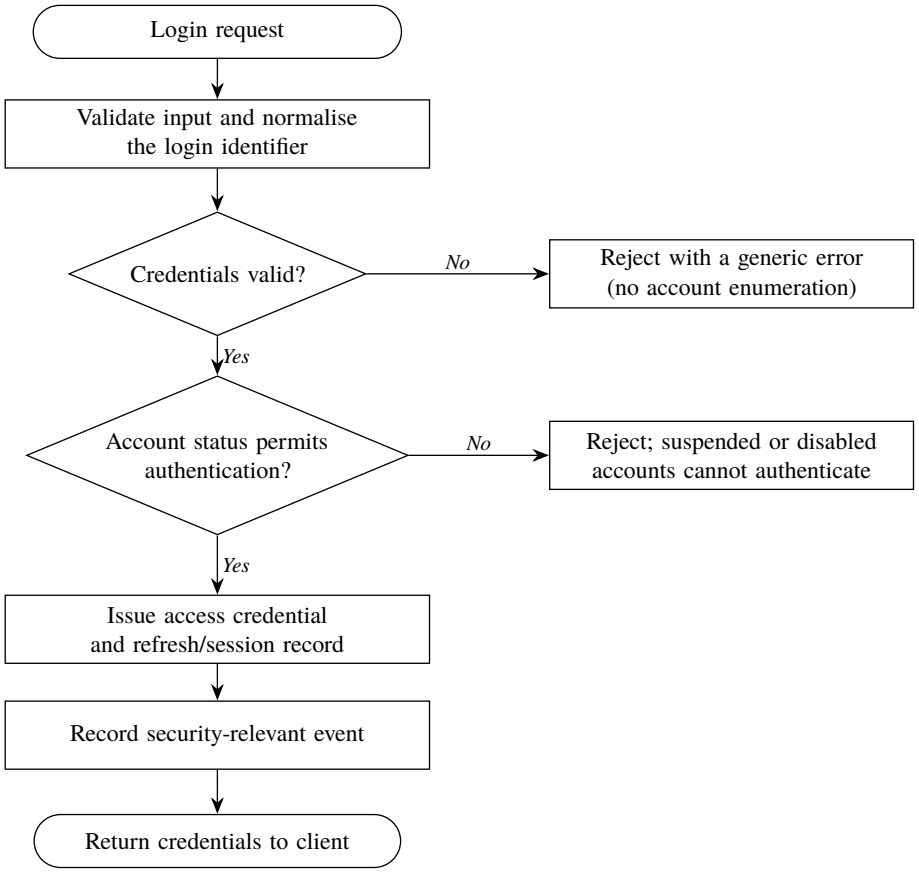


Figure 6. Login flow.

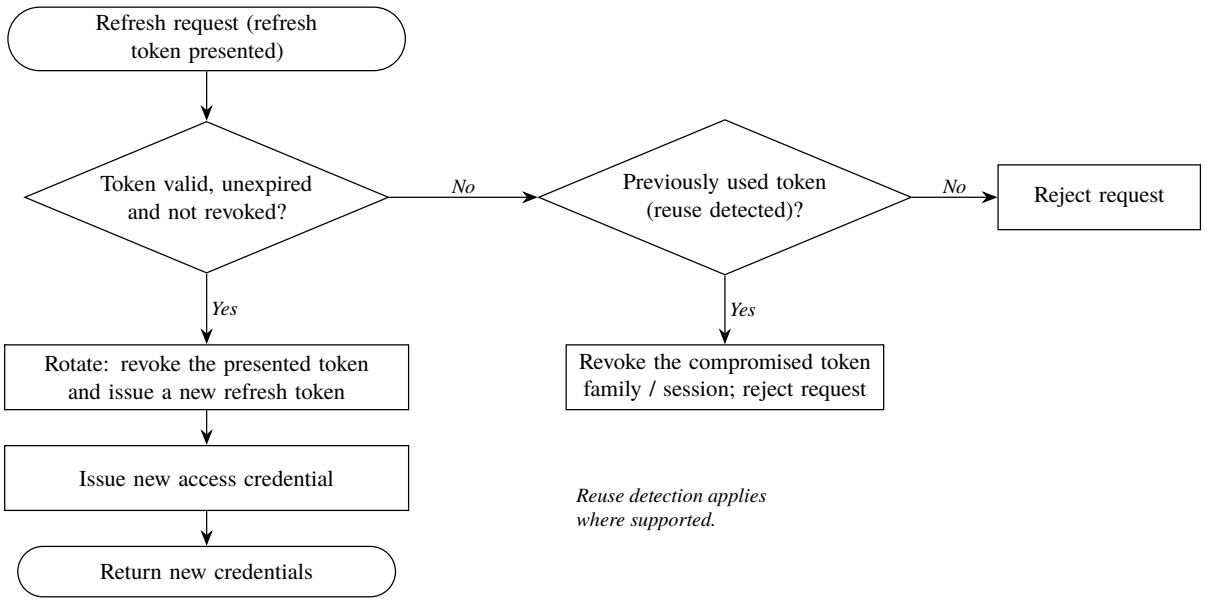


Figure 7. Refresh-token rotation and reuse detection.

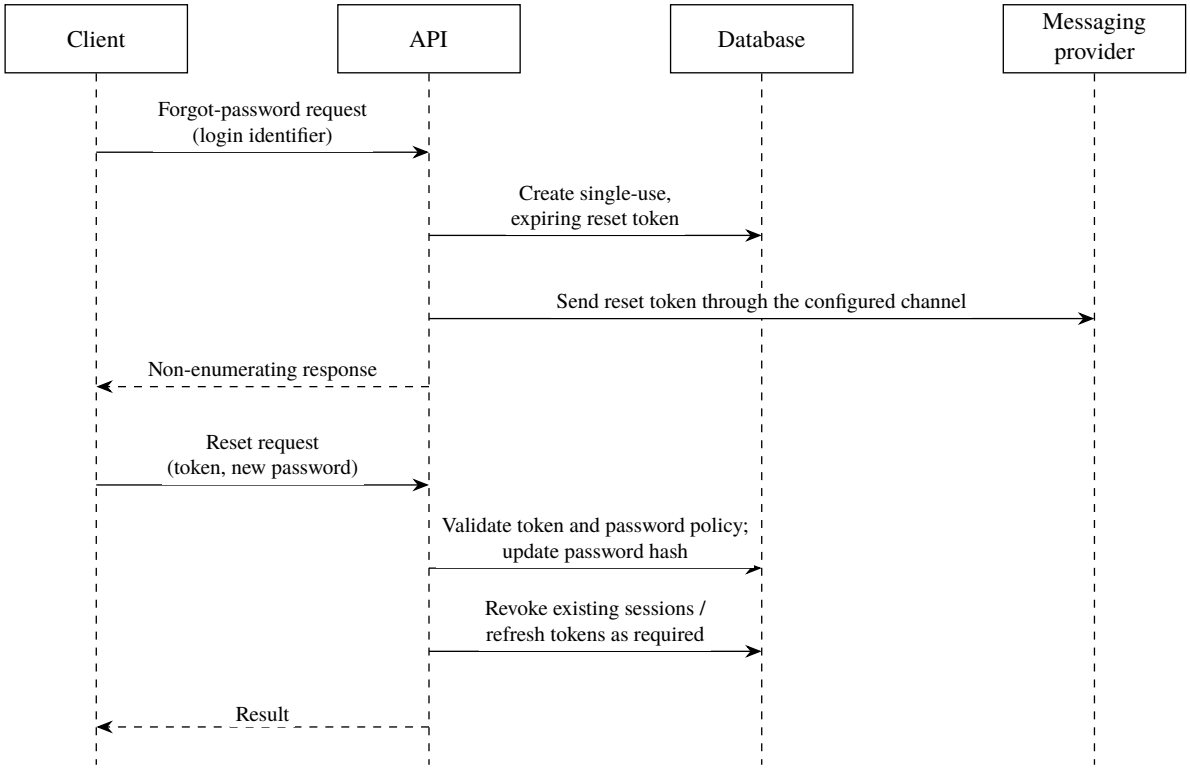


Figure 8. Password recovery and reset sequence.

8.4 Account and approval statuses

Table 19. Account status transitions

From	To	Trigger
(new)	PENDING_ VERIFICATION	Registration completed
PENDING_ VERIFICATION	ACTIVE	Verification code confirmed (customers); verification and administrator approval (merchants and riders)
ACTIVE	SUSPENDED	Administrator action with a reason; all sessions are revoked
SUSPENDED	ACTIVE	Administrator reactivation
ACTIVE or SUSPENDED	DISABLED	Administrator action or account closure request; sessions revoked; data retained per policy
DISABLED	ACTIVE	Administrator reactivation only

Suspended and disabled accounts must not authenticate or perform protected actions. Public visibility additionally depends on the entity-specific status: a merchant is listed only when `merchants.status` is `ACTIVE` (the other values are `PENDING_APPROVAL`, `SUSPENDED` and `CLOSED`), and a rider receives offers only when `riders.verification_status` is `APPROVED`.

9 API requirements and conventions

9.1 General conventions

Table 20. API conventions

Topic	Rule
Base path	`/api/v1`; a breaking change creates a new version path
Format	JSON request and response bodies, UTF-8, `Content-Type: application/json`
Naming	`camelCase` fields, `kebab-case` paths, plural nouns for collections, standard HTTP methods
Authentication	`Authorization: Bearer <accessToken>` (or session cookie for browsers) on every endpoint except those marked Public
Identifiers and dates	UUID strings; ISO 8601 timestamps in UTC
Money	JSON numbers in major units with two decimals plus a `currency` code. The client never sends prices or totals; the server calculates them
Pagination	`page` and `limit` query parameters (default 20, maximum 100) with `meta.page`, `meta.limit`, `meta.total` and `meta.totalPages`; cursor pagination is used for high-volume lists such as notifications and tracking
Sorting and filtering	`sort=field` or `sort=-field` (descending) and documented filters per endpoint; unknown parameters are rejected
Validation	Every path, query and body parameter is validated with Zod; failures return `VALIDATION_ERROR` with field-level details
Idempotency	`Idempotency-Key` header required on `POST /orders` and `POST /orders/{id}/payments`; a repeated key with the same body returns the original result, with a different body returns `IDEMPOTENCY_CONFLICT`
Documentation	OpenAPI 3.1 generated from the Zod schemas and served through Swagger UI; contract tests verify that implemented endpoints match the specification
Models	Internal database models are never exposed directly; responses use explicit response schemas

9.2 Common response shape

Success:

```
{
  "data": {},
  "meta": {
    "requestId": "request-id"
  }
}
```

Paginated lists add `page`, `limit`, `total` and `totalPages` to `meta`. Error:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request could not be processed.",
    "details": [{ "field": "email", "message": "Must be a valid email address"
     }],
    "requestId": "request-id"
  }
}
```

9.3 Error codes

Table 21. Error codes

HTTP	Code	When
400	`BAD_REQUEST`	Malformed JSON or parameters
401	`UNAUTHENTICATED`, `TOKEN_EXPIRED`	Missing, invalid or expired credentials; the client refreshes once
402	`PAYMENT_FAILED`	Gateway declined or verification failed
403	`FORBIDDEN`, `ACCOUNT_PENDING`, `ACCOUNT_SUSPENDED`	Role or status check failed
404	`NOT_FOUND`	Resource missing, or not visible to this principal
409	`CONFLICT`	Duplicate resource such as an existing email address
409	`INVALID_STATE_TRANSITION`	Status change not allowed from the current status
409	`OUT_OF_STOCK`, `MERCHANT_CLOSED`, `PRODUCT_UNAVAILABLE`	Order cannot be fulfilled as requested
409	`IDEMPOTENCY_CONFLICT`	Idempotency key reused with a different body
422	`VALIDATION_ERROR`, `BELOW_MIN_ORDER`, `OUTSIDE_DELIVERY_AREA`	Field validation or business rule failed
429	`RATE_LIMITED`	Too many requests; `Retry-After` header is returned
500	`INTERNAL_ERROR`	Unexpected failure; no internal details are exposed

9.4 Rate limits

Table 22. Default rate limits

Endpoint group	Limit	Key
Login	10 per 15 minutes	IP and identifier
Registration, verification resend, password recovery	5 per hour	IP and identifier
Public discovery	120 per minute	IP
Authenticated endpoints	300 per minute	User
Rider location updates	30 per minute	User
Payment webhooks	Not limited; signature required	Provider

9.5 Endpoint catalogue

The tables below list every endpoint of the first release. Access codes: **P** public, **C** customer, **M** merchant (owner or staff), **R** rider, **A** administrator. Paths are relative to `/api/v1`.

Table 23. Authentication endpoints

Method and path	Access	Purpose
`POST /auth/register`	P	Create a customer, merchant or rider account
`POST /auth/verify`	P	Confirm the email or phone verification code
`POST /auth/resend-verification`	P	Send a new code (rate limited)
`POST /auth/login`	P	Exchange credentials for access and refresh tokens
`POST /auth/refresh`	P	Rotate the refresh token and obtain a new access token
`POST /auth/logout`	Any	Revoke the current session
`POST /auth/forgot-password`	P	Start password recovery (always returns 200)
`POST /auth/reset-password`	P	Set a new password with a valid token; revokes all sessions
`POST /auth/change-password`	Any	Change password using the current password
`GET /auth/sessions`	Any	List own active sessions
`DELETE /auth/sessions/{sessionId}`	Any	Revoke one session

Table 24. User and profile endpoints

Method and path	Access	Purpose
`GET /users/me`, `PATCH /users/me`	Any	Read and update own account and role profile
`GET /users/me/addresses`	C	List saved addresses
`POST /users/me/addresses`	C	Add an address (geocoded on save)
`PATCH /users/me/addresses/{addressId}`, `DELETE ...`	C	Update or remove an address; set default
`POST /users/me/device-tokens`	Any	Register a push token
`DELETE /users/me/device-tokens/{token}`	Any	Remove a push token
`POST /uploads/presign`	Any	Obtain a presigned upload URL for an image or document
`GET /admin/users`, `GET /admin/users/{userId}`	A	Search and view users (filters: role, status, q)
`PATCH /admin/users/{userId}/status`	A	Activate, suspend or disable with a reason
`POST /admin/admins`	SUPER_ADMIN	Create an administrator account

Table 25. Merchant discovery and catalogue endpoints (public)

Method and path	Access	Purpose
`GET /categories?type=MERCHANT` or `type=PRODUCT`	P	Active categories in sort order
`GET /merchants`	P	List and search active merchants. Filters: `q`, `categoryId`, `city`, `isOpen`, `deliversTo` (coordinates); `lat` and `lng` enable `sort=distance`; other sorts: `rating`, `name`
`GET /merchants/{idOrSlug}`	P	Merchant profile, hours, delivery zones and open-now flag
`GET /merchants/{merchantId}/products`	P	Published products of a merchant. Filters: `categoryId`, `q`; paginated
`GET /products`	P	Search published products across active merchants. Filters: `q`, `categoryId`, `merchantId`
`GET /products/{productId}`	P	Product detail with option groups, price and availability

Table 26. Merchant management endpoints

Method and path	Access	Purpose
`POST /merchants`	M	Submit a merchant application (status PENDING_APPROVAL)
`GET /merchant/profile`, `PATCH /merchant/profile`	M	Read and update own merchant profile
`PATCH /merchant/status`	M	Open or close the shop (`isOpen`)
`PUT /merchant/hours`	M	Replace the weekly opening hours
`GET/POST /merchant/delivery-zones`	M	List and create delivery zones
`PATCH/DELETE /merchant/delivery-zones/{zoneId}`	M	Update or remove a zone
`GET/POST /merchant/staff`, `DELETE /merchant/staff/{userId}`	M (owner)	Manage staff accounts
`GET/POST /merchant/products`	M	List (including drafts) and create products
`GET/PATCH/DELETE /merchant/products/{productId}`	M	Read, update and soft-delete a product
`PATCH /merchant/products/{productId}/status`	M	Publish, unpublish (DRAFT) or archive
`PATCH /merchant/products/{productId}/availability`	M	Sold-out switch
`PUT /merchant/products/{productId}/option-groups`	M	Replace option groups and options
`PUT /merchant/products/{productId}/inventory`	M	Set stock level and threshold
`POST /merchant/products/{productId}/inventory/adjustments`	M	Record a stock movement
`GET /merchant/products/{productId}/inventory/movements`	M	Stock history
`GET /merchant/orders`, `GET /merchant/orders/{orderId}`	M	Order queue (filter by status) and detail
`PATCH /merchant/orders/{orderId}/status`	M	ACCEPTED, REJECTED (reason required), PREPARING, READY_FOR_PICKUP, COMPLETED (pickup orders)
`GET /merchant/settlements`	M	Ledger entries and payable balance

Table 27. Order and payment endpoints (customer)

Method and path	Access	Purpose
`POST /orders/quote`	C	Validate and price a basket: subtotal, delivery fee, total, availability and delivery eligibility
`POST /orders`	C	Place an order (`Idempotency-Key` required)
`GET /orders`, `GET /orders/{orderId}`	C	Own order history and detail
`POST /orders/{orderId}/cancel`	C	Cancel while the order is PENDING_PAYMENT or PLACED
`GET /orders/{orderId}/tracking`	C	Status history, delivery status, latest rider location and estimate
`POST /orders/{orderId}/payments`	C	Start an online payment (`Idempotency-Key` required)
`GET /payments/{paymentId}`	C, A	Payment status
`POST /webhooks/paystack`	P (signed)	Payment provider event receiver

Table 28. Rider and delivery endpoints

Method and path	Access	Purpose
`POST /rider/documents`	R	Register an uploaded verification document
`PATCH /rider/availability`	R	Go AVAILABLE or OFFLINE
`POST /rider/location`	R	Send a location point
`GET /rider/offers`	R	Current open delivery offer, if any
`POST /rider/offers/{assignmentId}/accept`, `.../reject`	R	Respond to an offer
`GET /rider/deliveries`, `GET /rider/deliveries/{deliveryId}`	R	Active and past deliveries
`POST /rider/deliveries/{deliveryId}/arrived`	R	Arrived at the merchant (AT_PICKUP)
`POST /rider/deliveries/{deliveryId}/pickup`	R	Order collected (PICKED_UP); allowed only when the order is READY_FOR_PICKUP
`POST /rider/deliveries/{deliveryId}/deliver`	R	Complete with the customer confirmation code and optional proof photo
`POST /rider/deliveries/{deliveryId}/fail`	R	Report a failed delivery with a reason
`GET /rider/earnings`	R	Earnings summary and history

Table 29. Notification endpoints

Method and path	Access	Purpose
`GET /notifications`	Any	Own in-app notifications (cursor paginated; filter `unread`)
`PATCH /notifications/{notificationId}/read`	Any	Mark one notification read
`POST /notifications/read-all`	Any	Mark all notifications read
`GET /notifications/preferences`, `PUT /notifications/preferences`	Any	Read and replace channel and category preferences

Table 30. Administration and system endpoints

Method and path	Access	Purpose
`GET /admin/merchants`, `GET /admin/merchants/{merchantId}`	A	Review merchants (filter by status)
`PATCH /admin/merchants/{merchantId}/status`	A	Approve, suspend, reactivate or close
`GET /admin/riders`, `PATCH /admin/riders/{riderId}/verification`	A	Review documents; approve or reject a rider
`GET /admin/orders`, `GET /admin/orders/{orderId}`	A	All orders with filters
`PATCH /admin/orders/{orderId}/status`	A	Override status with a reason (audited)
`GET /admin/payments`	A	Payments with filters (status, date, order)
`POST /admin/payments/{paymentId}/refunds`	A	Full or partial refund
`GET /admin/deliveries`	A	Monitor deliveries; filter unassigned and failed
`POST /admin/deliveries/{deliveryId}/assign`	A	Manually assign or reassign a rider
`GET/POST/PATCH/DELETE /admin/categories`	A	Manage merchant and product categories
`GET /admin/settings`, `PUT /admin/settings`	SUPER_ADMIN	Platform configuration (fees, timeouts, radius)
`GET /admin/reports/summary`	A	Orders, revenue, active merchants and riders for a date range
`GET /admin/audit-logs`	A	Search the audit trail
`POST /admin/cod-remittances`	A	Record cash remitted by a rider

Method and path	Access	Purpose
`GET /health`, `GET /ready`	P	Liveness and readiness probes

9.6 Core request and response structures

The examples below fix the shape of the principal contracts; all other endpoints follow the same conventions, and the full schemas are generated into the OpenAPI document.

Register (`POST /auth/register`), request:

```
{
  "role": "CUSTOMER",
  "email": "ama@example.com",
  "phone": "+233200000002",
  "password": "S3cure!pass",
  "firstName": "Ama",
  "lastName": "Mensah"
}
```

Register, response (201):

```
{
  "data": {
    "userId": "uuid",
    "status": "PENDING_VERIFICATION",
    "verification": { "channel": "SMS", "expiresInSeconds": 600 }
  },
  "meta": { "requestId": "request-id" }
}
```

Login (`POST /auth/login`), response (200):

```
{
  "data": {
    "accessToken": "jwt",
    "expiresIn": 900,
    "refreshToken": "opaque",
    "user": { "id": "uuid", "role": "CUSTOMER", "firstName": "Ama", "status": "
      ACTIVE" }
  },
  "meta": { "requestId": "request-id" }
}
```

Merchant list item (`GET /merchants`):

```
{
  "id": "uuid",
  "slug": "mama-efua-kitchen",
  "businessName": "Mama Efua Kitchen",
  "categories": [{ "id": "uuid", "name": "Restaurants" }],
  "isOpen": true,
  "rating": 4.6,
  "ratingCount": 128,
  "distanceKm": 2.4,
  "minOrderAmount": 30.00,
  "deliveryFeeFrom": 10.00,
  "estimatedMinutes": 30
}
```

Place an order (`POST /orders`), request:

```
{
  "merchantId": "uuid",
  "fulfilment": "DELIVERY",
  "addressId": "uuid",
  "paymentMethod": "MOBILE_MONEY",
  "items": [
    { "productId": "uuid", "quantity": 2, "optionIds": ["uuid"], "note": "Less
      pepper" }
  ],
  "customerNote": "Call on arrival"
}
```

Place an order, response (201):

```
{
  "data": {
    "id": "uuid",
    "orderNumber": "EPD-261108-100001",
    "status": "PENDING_PAYMENT",
    "paymentStatus": "PENDING",
    "fulfilment": "DELIVERY",
    "items": [
      { "productName": "Jollof Rice", "unitPrice": 50.00, "quantity": 2, "
     lineTotal": 100.00 }
    ],
    "subtotal": 100.00,
    "deliveryFee": 10.00,
    "serviceFee": 2.00,
    "discountTotal": 0.00,
    "taxTotal": 0.00,
    "total": 112.00,
    "currency": "GHS",
    "expiresAt": "2026-11-08T10:30:00Z"
  }
}
```

Start payment (`POST /orders/{orderId}/payments`), request and response:

```
{ "method": "MOBILE_MONEY", "callbackUrl": "https://app.example.com/pay/return"
     }

{
  "data": {
    "paymentId": "uuid",
    "status": "PENDING",
    "amount": 112.00,
    "currency": "GHS",
    "reference": "EPD-PAY-8f2c1a",
    "authorizationUrl": "https://checkout.example/..."
  }
}
```

Delivery offer (`GET /rider/offers`):

```
{
  "data": {
    "assignmentId": "uuid",
    "deliveryId": "uuid",
    "expiresAt": "2026-11-08T10:15:45Z",
    "pickup": { "name": "Mama Efua Kitchen", "latitude": 5.556, "longitude":
     -0.183 },
    "dropoff": { "latitude": 5.570, "longitude": -0.175 },
    "distanceKm": 3.1,
    "estimatedMinutes": 18,
    "earning": 8.00
  }
}
```

Tracking (`GET /orders/{orderId}/tracking`):

```
{
  "data": {
    "orderStatus": "OUT_FOR_DELIVERY",
    "deliveryStatus": "PICKED_UP",
    "rider": { "firstName": "Kofi", "vehicleType": "MOTORBIKE" },
    "location": { "latitude": 5.562, "longitude": -0.179, "recordedAt":
     "2026-11-08T10:21:10Z" },
    "estimatedArrival": "2026-11-08T10:32:00Z",
    "confirmationCode": "4821"
  }
}
```

10 Core business workflows

10.1 Merchant and product discovery

Visibility rules. A merchant is publicly visible when `status = ACTIVE` and `deleted_at IS NULL`. A product is publicly visible when `status = PUBLISHED`, `deleted_at IS NULL` and its merchant is publicly visible. Draft, archived, deleted and pending items are never returned by public endpoints, and every public query applies these predicates in the repository, not in the controller.

Open and available. The `isOpen` flag returned to clients is true only when the merchant's manual switch (`is_open`) is on and the current time, evaluated in the merchant's `timezone`, falls inside one of the day's `merchant_hours`. Closed merchants remain listed with `isOpen = false` so customers can browse, but orders to them are refused with `MERCHANT_CLOSED`. A published product with `is_available = false` is returned as unavailable and cannot be ordered.

Search and filtering. `q` matches case-insensitively against merchant name, merchant category names and product name using trigram indexes (minimum two characters). `categoryId` filters by category, including child categories. `city` filters by merchant city. When `lat` and `lng` are supplied, `sort=distance` orders

by Haversine distance and `distanceKm` is returned. `deliversTo` (coordinates) keeps only merchants with an active delivery zone whose `max_distance_km` covers the distance, and returns the applicable delivery fee. The default sort is rating descending then name; every list is paginated with a stable secondary sort on `id`.

10.2 Order creation

1. Authenticate the customer, check the account is `ACTIVE` and check the `Idempotency-Key`; a repeated key returns the original order.
2. Load the merchant: it must be publicly visible, open, and accept the requested fulfilment type.
3. Load every product: each must belong to that merchant and be published and available; validate selected options against the option-group minimum and maximum; quantity per item must be between 1 and 50.
4. For `DELIVERY` orders load the customer's own address, compute the distance to the merchant and select the smallest active delivery zone that covers it; otherwise return `OUTSIDE_DELIVERY_AREA`.
5. Calculate prices on the server (formula below) and verify the merchant's minimum order amount (`BELOW_MIN_ORDER`).
6. In one transaction: insert the order with its item snapshots and the address snapshot, insert the first `order_status_history` row, reserve stock for tracked products, and write an outbox event.
7. Online-payment orders are created as `PENDING_PAYMENT` with `expires_at` set 15 minutes ahead; cash-on-delivery orders are created directly as `PLACED`.

Pricing formula. The final unit price is the product price plus the `price_delta` of the selected options. Then:

`line_total = unit_price x quantity`; `subtotal = sum of line_total`; `delivery_fee` = fee of the selected zone (zero for pickup); `service_fee = round(subtotal x SERVICE_FEE_RATE, 2)`; `total = subtotal + delivery_fee + service_fee + tax_total - discount_total`. Promotions are outside the first release (`discount_total = 0`) and `tax_total` follows a configured rate that defaults to zero.

Stock handling (D11). For products with `track_inventory = true`: order creation increases `quantity_reserved` (movement RESERVATION); reaching `PLACED` converts the reservation into a sale (`quantity_on_hand` and `quantity_reserved` both decrease; movement SALE); cancellation, rejection or expiry before that point releases the reservation (movement RELEASE). A cancellation after the sale is recorded as a RETURN movement only when the merchant confirms the goods were not prepared.

10.3 Order status lifecycle

Table 31. Order status transitions

From	To	Actor	Side effects and guards
(new)	PENDING_ PAYMENT	Customer (online payment)	Stock reserved; `expires_at` = now + 15 minutes
(new)	PLACED	Customer (cash on delivery)	Stock reserved and committed; merchant notified
PENDING_ PAYMENT	PLACED	System, on verified payment	Stock committed; CHARGE ledger entry; merchant and customer notified
PENDING_ PAYMENT	CANCELLED	Customer, or system at `expires_at`	Stock released
PLACED	ACCEPTED	Merchant	Delivery record created and dispatch started (delivery orders); customer notified
PLACED	REJECTED	Merchant, reason required	Stock released; automatic refund if paid; customer notified
PLACED	CANCELLED	Customer, or system if the merchant does not respond within 10 minutes	Stock released; automatic refund if paid
ACCEPTED	PREPARING	Merchant	Customer notified
PREPARING	READY_FOR_ PICKUP	Merchant	Assigned rider notified; pickup becomes allowed
READY_FOR_ PICKUP	OUT_FOR_ DELIVERY	Rider confirms pickup (delivery orders)	Confirmation code issued to the customer; tracking visible
READY_FOR_ PICKUP	COMPLETED	Merchant (pickup orders)	Ledger entries written
OUT_FOR_ DELIVERY	COMPLETED	Rider confirms delivery with the customer code	Rider earning and merchant payable entries; cash collection recorded for cash on delivery
ACCEPTED, PREPARING, READY_FOR_ PICKUP, OUT_ FOR_DELIVERY	CANCELLED	Administrator only, reason required	Delivery cancelled; refund and stock handling decided by the administrator; all parties notified

Any other transition returns `INVALID_STATE_TRANSITION`. Every transition is a compare-and-set update inside a transaction that also writes `order_status_history` and an outbox event. Cash on delivery is available only for delivery orders. Figure 9 shows the lifecycle.

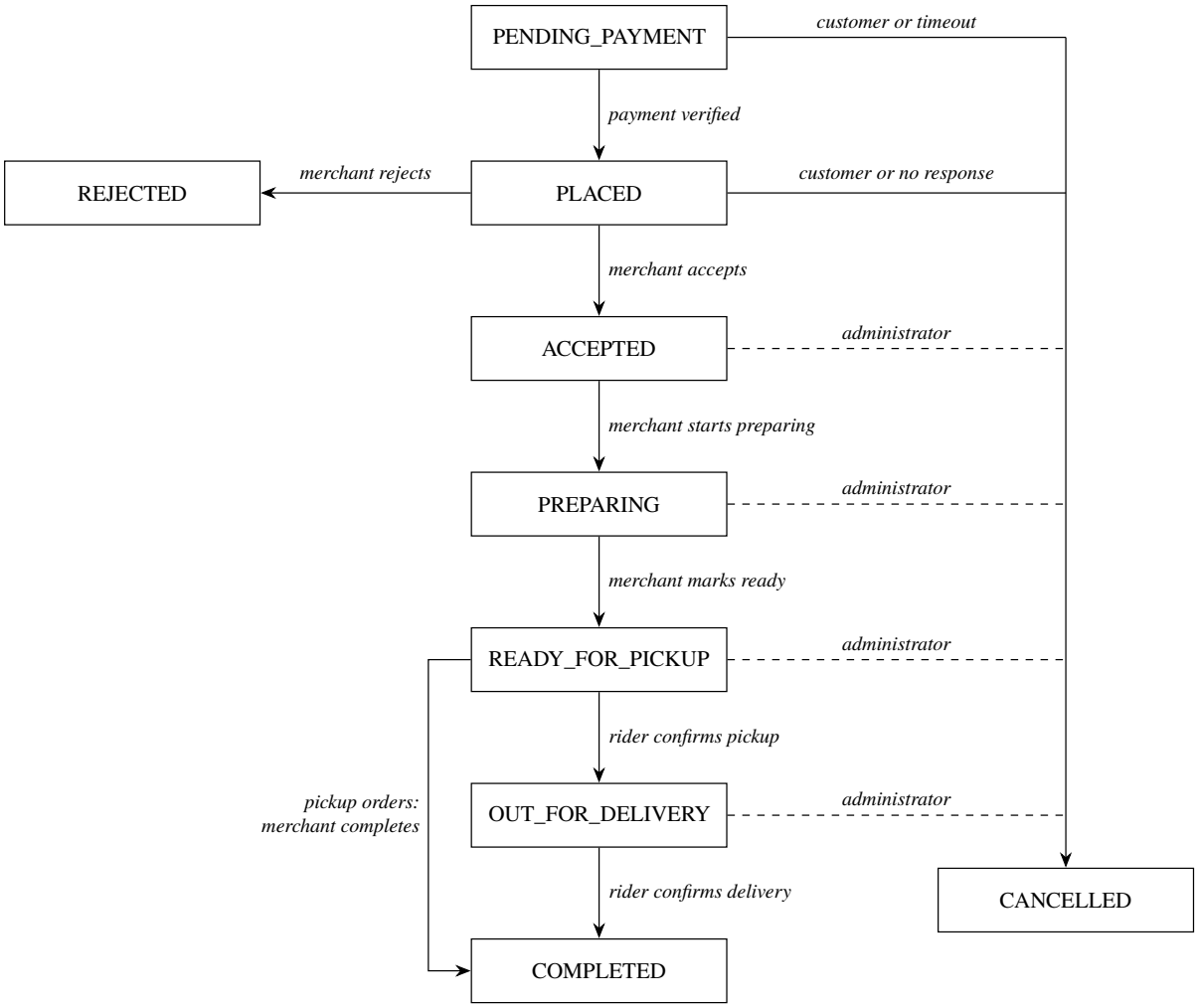


Figure 9. Order status lifecycle.

10.4 Payment processing

Payment model. A payment is one attempt to pay for one order. An order may have several payments (retries) but at most one with status `SUCCESS`. Online payments use the provider's hosted checkout (D3), so card and mobile-money details are entered on the provider's page and never reach EpaDash servers. The payment row stores the provider reference, the amount and currency calculated by the server, the idempotency key and a sanitised copy of the provider response. `orders.payment_status` always mirrors the latest relevant payment.

Table 32. Payment status transitions

From	To	Trigger
(new)	PENDING	Payment initiated for an order in PENDING_PAYMENT
PENDING	SUCCESS	Webhook or reconciliation verifies a successful transaction with matching reference, amount and currency
PENDING	FAILED	Provider reports failure, verification mismatch, or provider initialisation error
PENDING	CANCELLED	Customer abandons the payment and the order expires or is cancelled
SUCCESS	PARTIALLY_ REFUNDED	A refund smaller than the amount succeeds
SUCCESS or PARTIALLY_ REFUNDED	REFUNDED	Refunds succeed for the full amount

Online payment sequence. Figure 10 shows the exchange. The client asks the API to start a payment; the API validates that the order belongs to the caller and is in `PENDING_PAYMENT`, creates a `PENDING` payment, initialises a transaction with the provider using a unique reference, and returns the checkout URL. The customer pays on the provider's page. The provider then calls the webhook endpoint; the API verifies the event and, in one database transaction, marks the payment `SUCCESS`, moves the order to `PLACED`, commits the stock reservation, writes the CHARGE ledger entry and writes the outbox events for notifications. The client learns the result by polling `GET /payments/{paymentId}` or from the push notification; the browser redirect back from the provider is never treated as proof of payment.

Webhook verification. The webhook endpoint reads the raw request body, verifies the provider signature (HMAC) before any processing, stores the event in `payment_webhook_events` (a repeated event ID is acknowledged and ignored), then confirms the transaction status directly with the provider and checks reference, amount and currency against the stored payment. Only then is state changed. The endpoint returns a success response quickly and does heavy work through the queue (Figure 11).

Reconciliation. A scheduled job runs every 5 minutes and verifies with the provider every payment that has been `PENDING` for more than 10 minutes, so a lost webhook never leaves a paid order unconfirmed. A daily job compares provider settlement records with the ledger and raises an administrator alert for any difference.

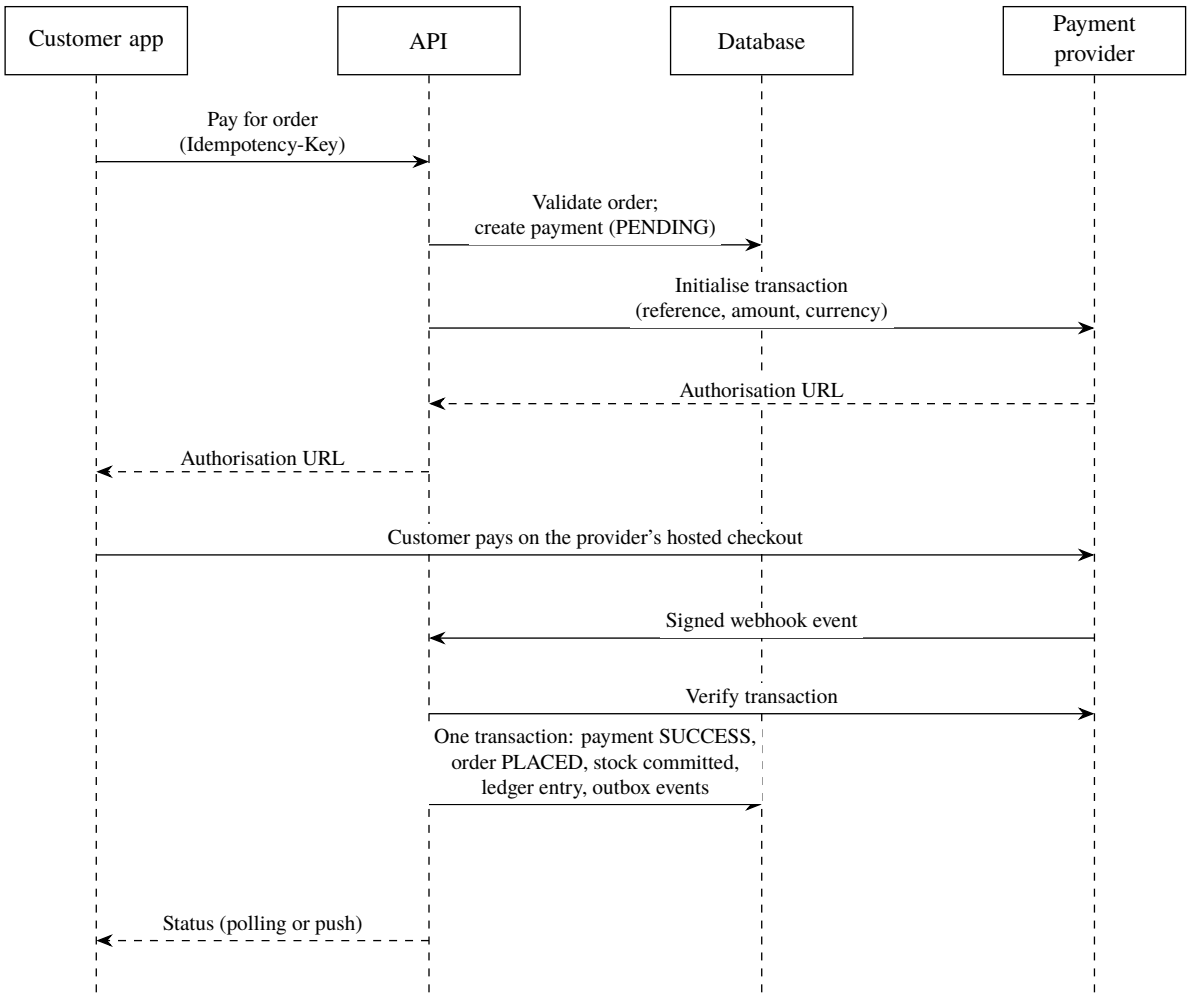


Figure 10. Online payment initiation and confirmation sequence.

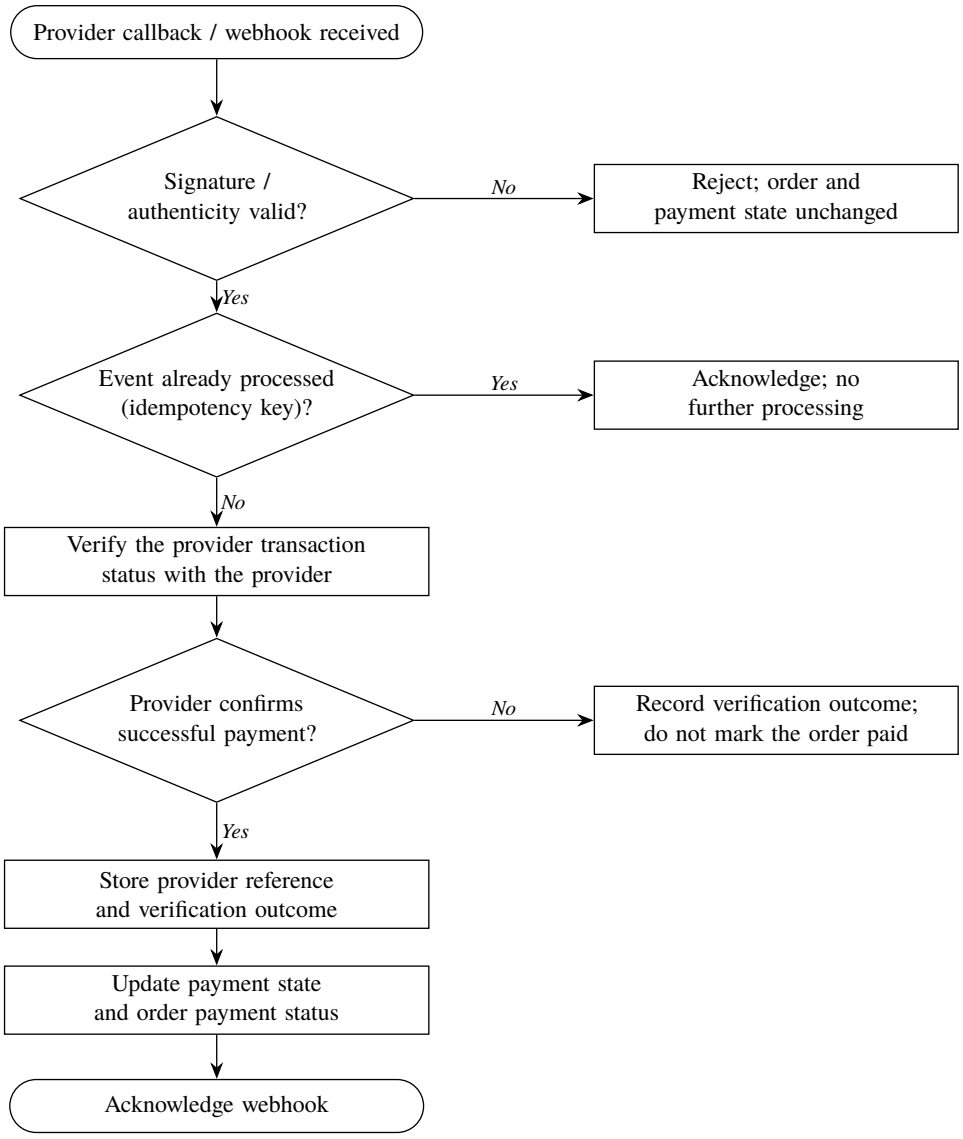


Figure 11. Payment webhook verification and idempotent processing.

Refunds. A full refund is created automatically when an order is rejected or cancelled after a successful payment. Administrators can issue full or partial refunds with a reason. The service creates a `refunds` row (`PENDING`), calls the provider refund API, and on success writes a REFUND ledger entry and updates the payment status; failures are retried three times with back-off and then appear in the administrator queue. The sum of successful refunds can never exceed the payment amount.

Ledger and settlement. Every money movement is an immutable row in `transactions`. Amounts owed are tracked by credit and debit rows, so correcting or paying out never edits history.

Table 33. Ledger entries

Event	Ledger entries	Notes
Payment succeeds	CHARGE, CREDIT, order total	Provider reference stored
Order COMPLETED	MERCHANT_PAYOUT, CREDIT, `subtotal - commission`; RIDER_PAYOUT, CREDIT, `rider_earning`; PLATFORM_FEE, CREDIT, remainder	Commission is `subtotal x merchants.commission_rate`; the three entries sum to the CHARGE amount when tax and discount are zero
Refund succeeds	REFUND, DEBIT, refund amount	Linked to the refund and payment
Funds paid to a merchant or rider	MERCHANT_PAYOUT or RIDER_PAYOUT, DEBIT, amount paid	Payable balance is credits minus debits
Cash collected on delivery	COD_COLLECTION, CREDIT, order total	Rider owes the platform this cash
Rider remits cash	COD_COLLECTION, DEBIT, amount remitted	Recorded by an administrator; rider earning may be netted

In the first release the system calculates and records payable balances; disbursing funds is an operational finance step (manual transfer or the provider's payout API, which is an extension point of `PaymentProvider`).

Cash on delivery. A payment row with method `CASH_ON_DELIVERY` and status `PENDING` is created when the order is placed. When the rider confirms delivery, the payment becomes `SUCCESS` and the COD_COLLECTION entry is written. If the delivery fails or the order is cancelled, the payment becomes `CANCELLED`.

Failure handling.

Table 34. Payment failure handling

Scenario	Handling
Customer abandons the payment	At `expires_at` the worker cancels the payment and the order and releases the stock
Payment declined	Payment becomes FAILED; the order stays PENDING_PAYMENT until it expires and the customer may start a new payment
Webhook lost or delayed	Reconciliation job verifies the payment with the provider
Duplicate or replayed webhook	Unique (provider, event ID) makes processing idempotent; the event is acknowledged without change
Amount, currency or reference mismatch	Payment becomes FAILED with a mismatch reason, an alert is raised and the order is not placed
Payment succeeds after the order was cancelled or expired	An automatic full refund is created and the customer is notified
Provider unavailable at initialisation	`PAYMENT_FAILED` is returned and no payment is left pending; the customer retries
Refund fails	Retried with back-off, then escalated to the administrator queue
New payment on an already paid order	Rejected with `CONFLICT`

Security controls. Card data is never stored or logged; the integration remains within the provider-hosted, tokenised flow. Provider secrets are held in the secret store and differ between test and live environments. Webhook signatures are verified on the raw body, and the provider's published source addresses may additionally be allow-listed. Amounts are always calculated server-side. Payment initiation and refunds are audited.

10.5 Delivery management

Delivery record. For delivery orders a `deliveries` row is created when the merchant accepts the order. It stores the pickup coordinates (merchant), the drop-off coordinates (address snapshot), the distance, the delivery fee and the rider earning. Pickup orders never create a delivery.

Table 35. Delivery status transitions

From	To	Actor	Guard and effect
(new)	UNASSIGNED	System	Created when the order is ACCEPTED; dispatch starts
UNASSIGNED	ASSIGNED	Rider accepts an offer, or administrator assigns	Delivery row locked; rider set; rider becomes BUSY

From	To	Actor	Guard and effect
ASSIGNED	UNASSIGNED	Rider cancels, or administrator reassigns	Assignment closed as CANCELLED; rider released; dispatch restarts
ASSIGNED	AT_PICKUP	Assigned rider	Arrival at the merchant recorded
AT_PICKUP	PICKED_UP	Assigned rider	Only when the order is READY_FOR_ PICKUP; order becomes OUT_FOR_ DELIVERY
PICKED_UP	DELIVERED	Assigned rider	Correct confirmation code; order becomes COMPLETED; rider becomes AVAILABLE
PICKED_UP	FAILED	Assigned rider	Reason required; administrator decides the outcome
UNASSIGNED, ASSIGNED, AT_ PICKUP	CANCELLED	System or administrator	Order cancelled; rider released and notified

Dispatch algorithm (D12). Figure 12 shows the exchange.

1. A candidate rider must be `APPROVED`, have availability `AVAILABLE`, have a location newer than 120 seconds, be within the search radius of the merchant (5 km, widened to 8 km after the first round), and have no open offer or active delivery.
2. Candidates are ranked by distance to the merchant using the Haversine distance; the travel time of the best three may be refined with the mapping provider. Ties are broken by the longest time since the rider's last delivery.
3. The best candidate receives an offer: a `delivery_assignments` row (`OFFERED`) that expires after 45 seconds, delivered by push notification and visible through `GET /rider/offers`.
4. On accept, one transaction locks the delivery row (`FOR UPDATE`), verifies it is still `UNASSIGNED` and the offer is still open, sets the rider, marks the offer `ACCEPTED`, sets the rider `BUSY` and writes the outbox events. A second rider answering late receives `INVALID_STATE_TRANSITION`.
5. On reject or expiry the offer is closed and the next candidate is offered, up to five offers per round. If no one accepts, the worker retries every 60 seconds for 10 minutes and then places the delivery in the administrator queue for manual assignment or cancellation.

Location and tracking. While a rider is `AVAILABLE` the app sends a location every 30 seconds; during an active delivery every 5 seconds. Each update is validated (coordinate ranges, accuracy, plausible speed), written to `riders.current_*` and, during a delivery, appended to `delivery_tracking`. Customers read the latest point and estimated arrival through `GET /orders/{orderId}/tracking` (version 1 polling, D10). A rider with no location for 5 minutes during an active delivery raises an alert.

Pickup and handover. A rider may mark `AT_PICKUP` on arrival but cannot mark `PICKED_UP` until the merchant has set the order to `READY_FOR_PICKUP`. When the order becomes `OUT_FOR_DELIVERY` a four-digit confirmation code is generated, stored hashed in `deliveries` and shown to the customer; the rider

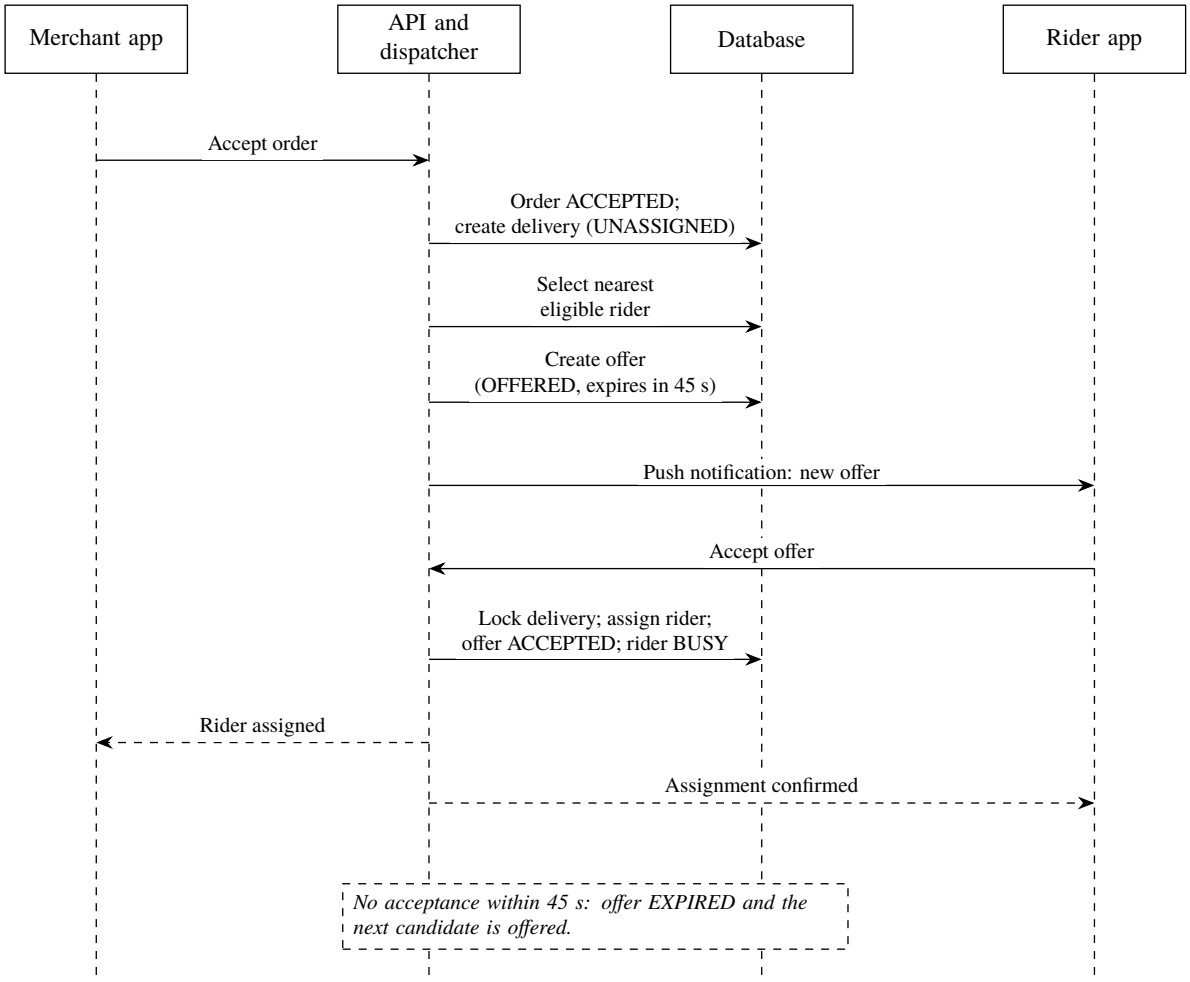


Figure 12. Rider dispatch and assignment sequence.

must enter it to complete the delivery, and an optional proof photo can be attached. Five wrong attempts flag the delivery for administrator review.

Delivery fee and rider earning. The customer's delivery fee is the fee of the smallest covering delivery zone (set at order creation and stored on the order and delivery). The rider earning is `delivery_fee x RIDER_FEE_SHARE` (default 80 percent); both values are platform settings.

Delivery configuration defaults.

Table 36. Delivery configuration defaults (platform settings)

Setting	Default	Notes
Offer expiry	45 seconds	Per offer
Search radius	5 km, then 8 km	Around the merchant
Offers per round	5	Then retry every 60 seconds for 10 minutes
Location freshness for dispatch	120 seconds	Rider must be recent
Location interval	30 s available, 5 s on delivery	Client behaviour
Maximum concurrent deliveries per rider	1	
Confirmation attempts	5	Then flagged
Customer wait before a failed delivery may be reported	10 minutes	
Rider fee share	80 percent of the delivery fee	

Failure, cancellation and reassignment.

Scenario	Handling

Table 37. Delivery exceptions

Scenario	Handling
Rider rejects or the offer expires	Offer closed as REJECTED or EXPIRED; the next candidate is offered
No rider accepts	Delivery stays UNASSIGNED, appears in the administrator queue; administrator assigns manually or cancels the order
Rider cancels after accepting	Assignment becomes CANCELLED, delivery returns to UNASSIGNED, dispatch restarts; the event is recorded and repeated cancellations are flagged
Rider stops sending location during a delivery	Alert after 5 minutes; administrator may reassign
Customer unreachable at drop-off	After the waiting period the rider reports FAILED with a reason; the order stays OUT_FOR_DELIVERY until the administrator decides redelivery, return or refund
Order cancelled by an administrator during delivery	Delivery becomes CANCELLED; rider released and notified
Wrong confirmation code	Attempt counted; after five attempts the delivery is flagged

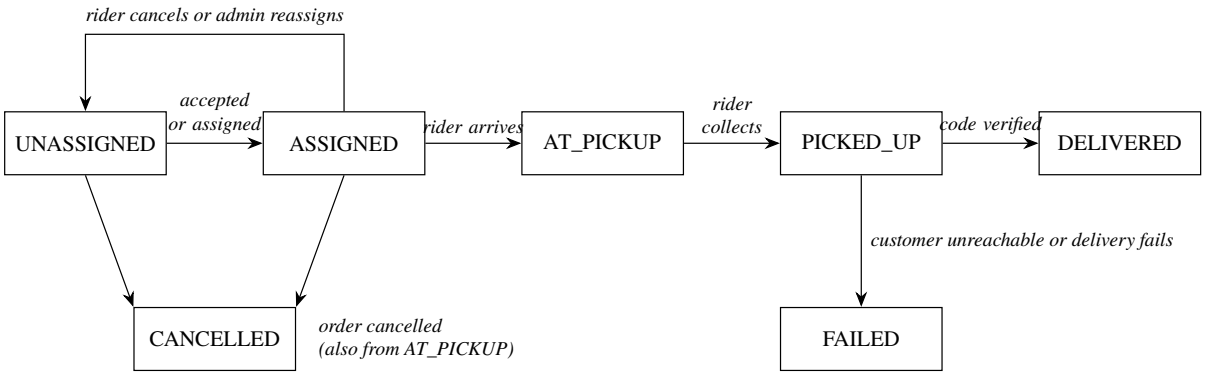


Figure 13. Delivery status lifecycle.

10.6 Notifications

Table 38. Notification events

Event	Recipients	Channels
Verification code, password reset	User	SMS or email (always sent)
Order placed	Merchant, customer	Push, in-app
Order accepted, preparing, ready, rejected, cancelled	Customer	Push, in-app; SMS for rejection and cancellation

Event	Recipients	Channels
Delivery offer	Rider	Push, in-app
Rider assigned, picked up, delivered	Customer, merchant	Push, in-app
Payment success, failure, refund	Customer	Push, email
Merchant approved or suspended	Merchant owner	Email, in-app
Rider verified or rejected	Rider	SMS, in-app
Delivery unassigned or failed, payment mismatch	Administrators	In-app, email

Domain changes write an `outbox_events` row in the same transaction as the change. The worker reads pending events, creates `notifications` rows for each recipient and channel allowed by the user's preferences (security messages ignore preferences), and sends them through the provider adapters. Failed sends are retried five times with exponential back-off (30 seconds, 2 minutes, 10 minutes, 30 minutes, 2 hours) and then marked `FAILED`; invalid push tokens are removed. Notification failure never changes order or payment state, and message payloads contain identifiers rather than sensitive data.

10.7 Administration and audit

Administrators approve and suspend merchants and riders, suspend and reactivate users, manage categories and platform settings, monitor and reassign deliveries, issue refunds, record cash remittances and view reports. The following actions always write an `audit_logs` row with the actor, role, entity, old and new values, IP address and user agent: account status changes, merchant and rider approval decisions, order status overrides, refunds, rider assignment overrides, platform setting changes, administrator creation and permission-sensitive reads of rider documents.

Figure 14 summarises the end-to-end workflow that the workflow tests in Section 13 exercise.

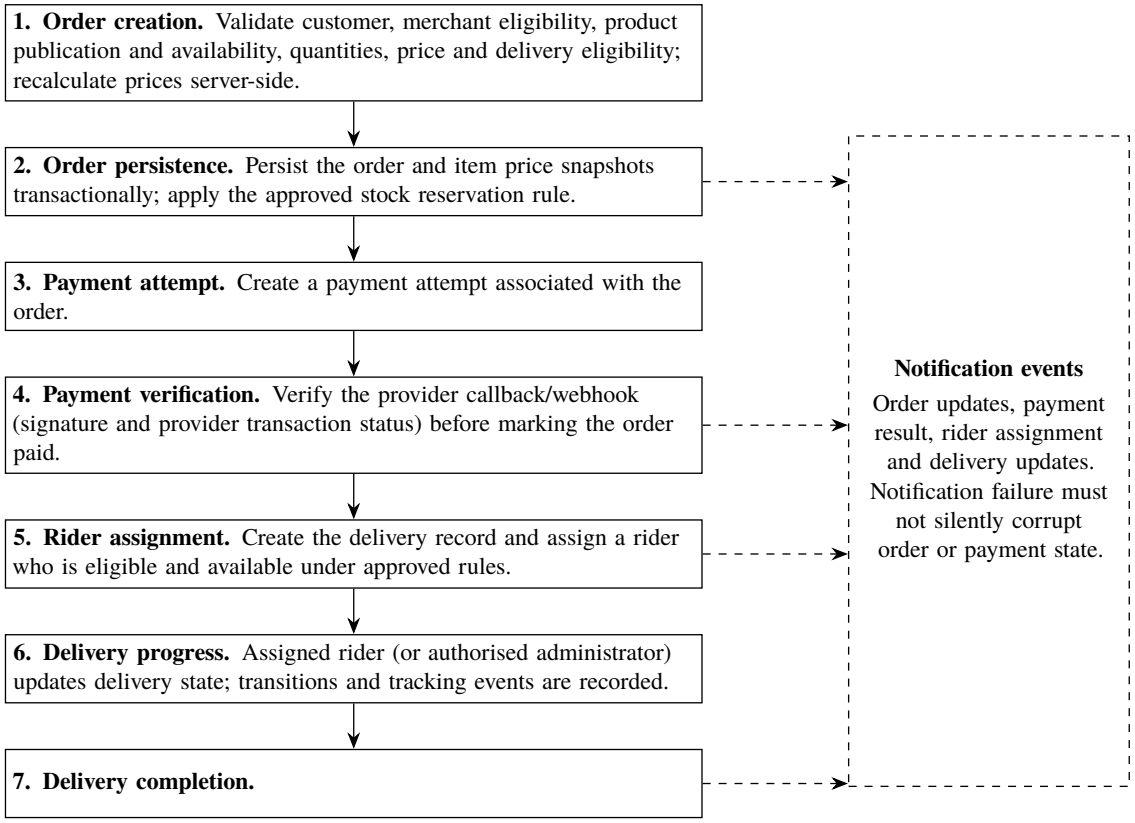


Figure 14. End-to-end workflow from order creation to delivery completion.

11 Third-party integration requirements

11.1 Integration summary

Table 39. Third-party integrations

Integration	Required capability	Boundary	Baseline decision
Payment provider	Initiate payment, verify status, receive signed webhooks, refund, support reconciliation	`PaymentProvider` adapter and provider-specific webhook verifier	Paystack hosted checkout (D3); sandbox and live keys per environment
Mapping and geolocation	Geocode addresses, distance and travel-time estimates, delivery-zone support	`MappingProvider` adapter; provider payloads never enter the domain model	Google Maps Platform (D4) with Haversine fallback
Notifications	Push, SMS and email for account and order events	`NotificationProvider` adapters called by the worker	Firebase Cloud Messaging for push; SMS and email providers selected per environment (D5)
File and media storage	Merchant and product images, rider documents	Storage adapter with presigned uploads and validated file metadata	S3-compatible object storage (D9)

Do not hard-code provider selection into business services. Use sandbox credentials during development,

keep credentials in secret configuration, verify webhooks cryptographically, and apply timeouts, retries with back-off and idempotency to every outbound call.

11.2 Adapter contracts

Each integration is consumed through a narrow interface, so a provider can be replaced without changing business logic or the API.

```
interface PaymentProvider {
  initializeTransaction(input: { reference: string; amountMinor: number;
      currency: string;
    email: string; callbackUrl: string; metadata?: Record<string, string> }):
    Promise<{ authorizationUrl: string; providerReference: string }>;
  verifyTransaction(reference: string):
    Promise<{ status: 'SUCCESS' | 'FAILED' | 'ABANDONED' | 'PENDING';
              amountMinor: number; currency: string; paidAt?: Date }>;
  verifyWebhookSignature(rawBody: Buffer, signature: string): boolean;
  parseWebhookEvent(rawBody: Buffer): { eventId: string; type: string; reference
      : string };
  refund(input: { reference: string; amountMinor: number; reason?: string }):
    Promise<{ providerRefundId: string; status: 'PENDING' | 'SUCCESS' | 'FAILED'
      }>;
}

interface MappingProvider {
  geocode(address: string):
    Promise<{ latitude: number; longitude: number; formattedAddress: string } |
     null>;
  travelEstimates(origin: Coordinates, destinations: Coordinates[]):
    Promise<{ distanceKm: number; durationMinutes: number }[]>;
}

interface NotificationProvider {
  channel: 'PUSH' | 'SMS' | 'EMAIL';
  send(message: { to: string; title?: string; body: string; data?: Record<string
      , string> }):
    Promise<{ providerMessageId: string }>;
}
```

11.3 Payment gateway requirements

- Hosted checkout supporting cards and mobile money, and a sandbox environment for automated and manual testing.
- Server-side transaction initialisation with a unique merchant reference, amount in minor units and currency, and a callback URL.
- Server-side transaction verification by reference, returning status, amount and currency.
- Signed webhooks for charge and refund events with a verifiable signature header and stable event

identifiers.

- Refund API (full and partial) and settlement reports for reconciliation.
- Separate test and live secret keys; keys are never exposed to clients or logs.
- The provider fee structure and settlement schedule are recorded in platform settings so that ledger reports can show them; the adapter is the only code that knows provider-specific field names.

11.4 Mapping requirements

- Geocode customer and merchant addresses when they are saved; store the coordinates with the address so that ordinary requests do not call the provider.
- Use provider travel estimates only to refine dispatch ranking and customer arrival estimates; delivery zone checks and fees use the stored coordinates and Haversine distance, so ordering still works if the provider is unavailable.
- Restrict API keys by service and origin, cache geocoding results, set quotas and alerts on usage, and keep provider payloads out of the domain model.
- Treat rider location as personal data: collect it only while a rider is available or on a delivery, and apply the retention period in Section 6.7.

11.5 Notification requirements

- Channels: in-app (database), push through Firebase Cloud Messaging for Android, iOS and web, SMS for verification codes and critical alerts, and email for receipts and account messages.
- Templates are stored in code with variables, versioned, and support localisation; sender identities (SMS sender ID, email domain with SPF and DKIM) are configured per environment.
- Delivery is asynchronous through the queue with retry and a stored delivery status (Section 10.6); user preferences apply except for security messages.
- Push token lifecycle: tokens are registered per device, refreshed on use and removed when the provider reports them invalid.

11.6 File and media storage

- Clients upload directly to object storage using presigned URLs obtained from `POST /uploads/presign`; the API never streams large files.
- Allowed image types and a maximum size (5 MB by default) are validated when the URL is issued and again when the object key is registered.
- Product and merchant images are served from a public path or CDN; rider verification documents are stored privately and read only through short-lived signed URLs by administrators.

11.7 Configuration and secrets

Table 40. Principal configuration variables

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
JWT_ACCESS_SECRET, ACCESS_TOKEN_TTL, REFRESH_TOKEN_TTL	Token signing secret and lifetimes
DEFAULT_CURRENCY	ISO 4217 currency code of the deployment
PAYSTACK_SECRET_KEY, PAYSTACK_WEBHOOK_SECRET	Payment provider credentials (test and live differ)
MAPS_API_KEY	Mapping provider credential
FCM_CREDENTIALS, SMS_PROVIDER_KEY, EMAIL_PROVIDER_KEY	Notification provider credentials
STORAGE_BUCKET, STORAGE_ACCESS_KEY, STORAGE_SECRET_KEY	Object storage
CORS_ALLOWED_ORIGINS	Approved frontend origins
SERVICE_FEE_RATE, RIDER_FEE_SHARE, ORDER_PAYMENT_TIMEOUT_MIN	Initial values for platform settings

All variables are validated with Zod at start-up; secrets are supplied by the environment or a secret manager and never committed.

12 Security, privacy and operational requirements

12.1 Security and privacy

- HTTPS/TLS outside local development, with HSTS.
- Strict request validation and safe output encoding.
- Authentication and resource-level authorisation on protected endpoints.
- Rate limiting and abuse protection for sensitive endpoints (Table 22).
- CORS allow-list based on approved frontend origins; no permissive wildcard configuration with credentials.
- Secure headers and regular dependency updates with automated vulnerability scanning.
- Environment-based secrets; least-privilege database credentials with separate migration and application roles; the application role cannot alter or delete ledger and audit rows.
- Structured logs with request IDs and redaction of secrets and personal data.
- Audit records for privileged actions and sensitive state transitions.
- Backups and restore testing appropriate to the hosting environment.

- Database migrations reviewed and run in a controlled deployment process.
- Avoid unnecessary personal data collection; retention and deletion follow Section 6.7 and the organisation's policy.
- Payment-card handling remains with a compliant provider-hosted, tokenised flow.
- Error messages must not disclose whether a particular account exists during password recovery or login.

12.2 Non-functional targets

Table 41. Initial non-functional targets

Attribute	Target
Availability	99.5 percent monthly for the API
Latency	95th percentile under 300 ms for read endpoints and under 800 ms for write endpoints, excluding calls to external providers
Initial sizing	5,000 orders per day and 500 riders reporting location (about 100 location writes per second at peak); to be validated by load testing before launch
Recovery	Recovery point objective of 15 minutes with point-in-time recovery; recovery time objective of 4 hours; restore tested every quarter
Security	OWASP API Security Top 10 controls verified by automated and manual tests before release
Observability	Request ID on every log line; error tracking; metrics for latency, error rate, queue depth, payment and dispatch outcomes; alerts on payment mismatches, stuck queues and unassigned deliveries

12.3 Environments and delivery process

- **Environments:** local (Docker Compose), automated test, staging and production, each with separate databases, secrets and provider sandbox or live credentials.
- **Continuous integration:** on every pull request run lint, type check, unit and integration tests against a PostgreSQL service, migration checks on an empty database, and OpenAPI validation.
- **Deployment:** build one container image from the repository; run migrations as a separate release step; run the API and worker processes from the same image; use health and readiness probes for rollout.
- **Operations:** daily automated backups with point-in-time recovery, log retention of 30 days online, and runbooks for payment reconciliation differences, stuck queues and unassigned deliveries.

13 Testing and quality strategy

- **Unit tests:** service rules, state transitions, permission decisions, price calculation, stock arithmetic and availability rules.
- **API integration tests:** routes, middleware, validation, status codes and response contracts against a real PostgreSQL test database.
- **Database tests:** migrations, constraints, transactions, partial unique indexes and repository behaviour against PostgreSQL.
- **Security tests:** unauthenticated access, forbidden role, cross-user access, cross-merchant access, token expiry and revocation, refresh reuse detection and rate-limit behaviour.
- **Provider tests:** adapter contract tests and sandbox tests; do not rely only on mocked provider responses.
- **Concurrency tests:** last-item stock contention, simultaneous rider acceptance, duplicate webhooks and duplicate order submissions.
- **Workflow tests:** order creation → payment verification → rider assignment → delivery completion, including rejection, cancellation, refund and failed-delivery paths.
- **Documentation checks:** the OpenAPI document validates and matches the implemented endpoints.

Acceptance evidence includes reproducible test commands and test results. A feature is not complete solely because its route exists. The target is at least 80 percent line coverage of service and domain code and a passing test for every transition in Table 31, Table 32 and Table 35.

14 Proposed repository structure

```
epadash-backend/
|-- docs/
|   |-- architecture/
|   |   |-- backend-technical-spec.md
|   |   `-- architecture-context.mmd
|   |-- database/
|   |   |-- database-design.md
|   |   `-- erd.mmd
|   |-- api/
|   |   `-- openapi.yaml
|   |-- workflows/
|   |-- decisions/
|   |   `-- ADR-0001-initial-architecture.md
|   `-- reports/
|-- prisma/
|   |-- schema.prisma
|   |-- migrations/
|   `-- seed.ts
|-- src/
|   |-- config/
|   |-- middleware/
|   |-- common/
|   |-- worker/
|   `-- modules/
|       |-- auth/
|       |-- users/
|       |-- merchants/
|       |-- products/
|       |-- inventory/
|       |-- orders/
|       |-- payments/
|       |-- riders/
|       |-- deliveries/
|       |-- notifications/
|       `-- admin/
|-- tests/
|-- .env.example
|-- .gitignore
|-- package.json
`-- README.md
```

This structure may be adjusted to company conventions without weakening module separation, testability or secret handling. Each module folder contains its routes, controller, service, repository, schemas and tests.

15 Implementation roadmap

The assignment tasks that follow this specification take their content directly from Section 6, Section 9 and Section 10: the detailed database deliverables (Prisma schema, migrations, seed scripts and ERD) and the API deliverables (OpenAPI file and workflow documents) are produced from this baseline without further architectural decisions.

Table 42. Implementation phases

Phase	Scope	Assignment task	Exit criteria
P0	Repository, CI, configuration, logging, error model, Prisma baseline, health endpoints	All	CI passes; migrations apply to an empty database
P1	Authentication, sessions, users, roles, RBAC, account lifecycle	Core services and authentication	Authentication and permission tests pass
P2	Categories, merchants, products, public discovery endpoints	Merchant discovery and catalogue	Visibility and search tests pass; OpenAPI published
P3	Quote, order creation, stock reservation, order lifecycle	Order processing	Order and concurrency tests pass
P4	Payment initiation, webhooks, reconciliation, refunds, ledger	Payments	Sandbox end-to-end payment passes; idempotency tests pass
P5	Rider onboarding, dispatch, tracking, handover, earnings	Delivery management	Dispatch and exception tests pass
P6	Outbox worker, notification providers, administration, audit, reports	Notifications and administration	Notification retry tests pass; audit coverage verified
P7	Load and security testing, backup restore test, documentation	Quality	Targets in Section 12.2 met

16 Delivery and approval checklist — Task 5

Table 43. Task 5 acceptance criteria and evidence

No.	Acceptance criterion from assignment	Evidence in this document	Status
1	Backend technology stack documented	Sections 2.3 and 3	Complete
2	Backend system architecture defined	Section 4	Complete
3	Core system modules identified	Section 5	Complete
4	Major entities and relationships outlined	Section 6 (36 entities, relationship matrix, diagrams)	Complete

No.	Acceptance criterion from assignment	Evidence in this document	Status
5	Customer, merchant, rider and administrator roles defined	Section 7	Complete
6	Authentication and authorisation documented	Sections 7 and 8	Complete
7	API requirements identified	Sections 9 and 10	Complete
8	Payment, mapping and notification integrations documented	Sections 10.4, 10.5 and 11	Complete
9	Technical specification submitted for approval	This document (revision 2)	**Submitted; approval pending**

Approval items

1. Approve the baseline decisions D1 to D13 in Table 3, or name the decision to be replaced.
2. Approve the default values of the platform settings (timeouts, radii, fee rates) or supply company values; they are configuration, not architecture.
3. Confirm hosting, environment, secrets, backup and deployment requirements.
4. Confirm data retention periods and any jurisdiction-specific compliance requirements.
5. Review and approve this technical specification.

Table 44. Approval record

Role	Name	Decision	Date
Technical lead			
Developer			
Frontend / API convention review			

Document status reminder: This is the revised technical specification for Task 5. It is not proof of approval, implementation, deployment or completed acceptance testing.